CSE 489 · Mobile Application Development · BRAC University

অ্যাপের ফিচার ও তাদের কার্যপদ্ধতি

Assignment 2: Fragments, Navigation Drawer ও Jetpack
বাংলায় তত্ত্বভিত্তিক ব্যাখ্যা

A **Broadcast Receiver**
নিজস্ব custom broadcast পাঠানো ও গ্রহণ, এবং সিস্টেমের ব্যাটারি broadcast শোনা

B **Image Scale**
ইন্টারনেট থেকে ছবি নামানো, pinch zoom, pan ও double-tap

C **Video**
VideoView ও MediaController দিয়ে অ্যাপের ভেতরে ভিডিও চালানো

D **Audio**
MediaPlayer দিয়ে গান: Play/Pause, Stop, seek slider ও সময়

এই সংস্করণ সম্পর্কে

- এটি ইংরেজি নির্দেশিকা `CSE489_Assignment2_Explained.pdf`-এর বাংলা সংস্করণ। এখানে কোনো ছবি বা কোডের তালিকা নেই; আছে অ্যাপের প্রতিটি ফিচার কী করে এবং ভেতরে কীভাবে কাজ করে, তার তত্ত্বভিত্তিক ব্যাখ্যা।
- Message Queue ও Publish/Subscribe আলাদা দুটি অধ্যায়ে গোড়া থেকে ব্যাখ্যা করা হয়েছে: দৈনন্দিন জীবনের উদাহরণ, সফটওয়্যারের উদাহরণ এবং এই অ্যাপের কোডে ধাপে ধাপে উদাহরণ।
- কোড ব্যাখ্যা করা প্রতিটি অংশে একটি "কোডে কোথায়" বাক্স বলে দেয় ব্যাখ্যাটি কোন ফাইলের কোন function-এর, কোন লাইনের। যেমন `CustomMessageReceiver.kt › onReceive() লাইন 23–28`।
- Activity, Intent, ViewModel-এর মতো প্রযুক্তিগত শব্দ ইংরেজিতেই রাখা হয়েছে, যাতে কোডের সঙ্গে মিলিয়ে পড়া যায় এবং viva-তে একই শব্দ ব্যবহার করা যায়।

সূচিপত্র

১ অ্যাপটি এক নজরে ৫

১.১ অ্যাসাইনমেন্টে কী চাওয়া হয়েছে ৫

১.২ অ্যাপের ফিচারসমূহ ৫

১.৩ ব্যবহৃত প্রযুক্তি ৬

১.৪ কোডের মানচিত্র ও “কোডে কোথায়” বাক্স ৬

২ Message Queue: ধারণা ও উদাহরণ ৮

২.১ সরাসরি ডাকার সমস্যা ৮

২.২ Message Queue কী ৮

২.৩ দৈনন্দিন উদাহরণ: রেস্তোরাঁ ও ব্যাংক ৯

২.৪ সফটওয়্যারের উদাহরণ: অনলাইন দোকান ১০

২.৫ Message queue-এর গুরুত্বপূর্ণ বৈশিষ্ট্য ১০

২.৬ Android-এর main thread: অ্যাপের ভেতরের message queue ১১

২.৭ এই অ্যাপে message queue-এর উদাহরণ ১১

২.৮ একটি বার্তার যাত্রা: Play চাপা থেকে slider চলা ১২

৩ Publish/Subscribe: ধারণা ও উদাহরণ ১৪

৩.১ Pub/Sub কী ১৪

৩.২ দৈনন্দিন উদাহরণ: YouTube, রেডিও ও নোটিশ বোর্ড ১৪

৩.৩ ধাপে ধাপে: একটি ক্রিকেট স্কোর অ্যাপ ১৫

৩.৪ Topic ও filter ১৫

৩.৫ তিন ধরনের decoupling ১৬

৩.৬ দেরিতে আসা subscriber: তিন রকম delivery ১৬

৩.৭ Observer pattern: একই প্রোগ্রামের ভেতরে pub/sub ১৭

৩.৮ Android-এর broadcast: একটি pub/sub ব্যবস্থা ১৭

৩.৯ উদাহরণ ১: custom broadcast-এর পুরো যাত্রা ১৮

৩.১০ উদাহরণ ২: ব্যাটারি, একটি retained বার্তা ১৯

৪ দুই ব্যবস্থার তুলনা ও এই অ্যাপের মানচিত্র ২০

৪.১ Message Queue বনাম Pub/Sub ২০

৪.২ Explicit Intent: point-to-point বার্তা ২০

৪.৩ বড় মানচিত্র: ধারণা থেকে কোড ২১

৪.৪ কোন ফিচারে কোন ব্যবস্থা ২২

৫ Lifecycle ও state ২৩

৫.১ Lifecycle: কখন subscribe, কখন unsubscribe ২৩

৫.২ Configuration change, ViewModel ও saved state ২৪

৬ অ্যাপের কাঠামো: Navigation Drawer ২৫

৬.১ একটি Activity, চারটি Fragment ২৫
৬.২ Navigation graph ও drawer menu-র সংযোগ ২৫
৬.৩ Fragment ও ViewBinding ২৬
৬.৪ Back চাপলে আগে drawer বন্ধ হয় ২৬
৬.৫ Edge-to-edge ও window insets ২৭

৭ ফিচার A: Broadcast Receiver ২৮
৭.১ পুরো প্রবাহ এক নজরে ২৮
৭.২ প্রথম স্ক্রিন: Spinner ও Proceed ২৮
৭.৩ Explicit Intent: এক Activity থেকে আরেক Activity-তে ২৯
৭.৪ Custom পথ, Activity 2: বার্তা নেওয়া ২৯
৭.৫ CustomMessageReceiver: আমাদের subscriber ৩০
৭.৬ Activity 3: register, publish, receive ৩১
৭.৭ হারিয়ে যাওয়া বার্তার সমস্যা ৩২
৭.৮ Rotation: বার্তা দুবার যায় না, কিছু হারায়ও না ৩২
৭.৯ Battery পথ: সিস্টেমের broadcast ৩৩
৭.১০ BatteryStatusLiveData: শুধু দৃশ্যমান থাকলে শোনা ৩৩
৭.১১ Battery Intent থেকে শতাংশ ও অবস্থা ৩৪
৭.১২ Battery পথ, Activity 3: কিছুই না করা ৩৫
৭.১৩ নিরাপত্তা: কে পাঠাতে পারে, কে শুনতে পারে ৩৫

৮ ফিচার B: Image Scale ৩৬
৮.১ ইন্টারনেট থেকে ছবি: Coil 3 ৩৬
৮.২ স্ক্রিনে state দেখানো ৩৬
৮.৩ Image Matrix ও “fit” ৩৭
৮.৪ Pinch zoom: ScaleGestureDetector ৩৭
৮.৫ এক আঙুলে pan ও কিনারা সামলানো ৩৮
৮.৬ Double-tap ৩৯
৮.৭ Touch event: একটি বার্তা-প্রবাহ ৩৯
৮.৮ Rotation-এ zoom ধরে রাখা ৪০

৯ ফিচার C: Video ৪১
৯.১ VideoView ও MediaController ৪১
৯.২ Player-এর তিনটি ঘটনা ৪১
৯.৩ থামানো ও মনে রাখা: onPause-এ কেন ৪২
৯.৪ VideoViewModel: অবস্থান কোথায় থাকে ৪২
৯.৫ Controls ও drawer-এর সমন্বয় ৪২

১০ ফিচার D: Audio ৪৪
১০.১ MediaPlayer একটি state machine ৪৪
১০.২ Player কেন ViewModel-এ ৪৪

১০.৩ State উপরে ওঠে, command নিচে নামে ৪৫
১০.৪ Progress: coroutine-এর একটি লুপ ৪৬
১০.৫ Slider ও সময়ের লেখা ৪৬
১০.৬ Stop, ট্র্যাক শেষ ও ত্রুটি ৪৬
১০.৭ অডিও ও ভিডিও কখনো একসঙ্গে বাজে না ৪৭

১১ সব স্ক্রিনে প্রযোজ্য বিষয় ৪৮
১১.১ Rotation: কোন স্ক্রিন কী ধরে রাখে ৪৮
১১.২ Theme, resource ও accessibility ৪৮

১২ দ্রুত পুনরালোচনা ৫০
১২.১ Message Queue ও Pub/Sub-এর মূল কথা ৫০
১২.২ প্রতিটি ফিচারের মূল কথা ৫১
১২.৩ পরিভাষা ৫২

১ অ্যাপটি এক নজরে

এই অধ্যায়ে দেখব অ্যাসাইনমেন্টে কী চাওয়া হয়েছিল, তৈরি অ্যাপটি কী কী করে, কোন প্রযুক্তি দিয়ে বানানো, এবং প্রজেক্টের কোন ফাইলে কী আছে।

১.১ অ্যাসাইনমেন্টে কী চাওয়া হয়েছে

CSE 489 (Mobile Application Development)-এর দ্বিতীয় অ্যাসাইনমেন্টের লক্ষ্য হলো Fragment, Navigation Drawer এবং Android Jetpack-এর ডিজাইন ও কোডিং দর্শন অনুশীলন করা। অ্যাপে একটি drawer menu থাকবে, যার চারটি item:

- **A. Broadcast Receiver:** প্রথম স্ক্রিনে একটি Spinner, যেখান থেকে broadcast-এর ধরন বেছে নিতে হয়: (১) Custom broadcast receiver, অথবা (২) System battery notification receiver। একটি বাটন চাপলে পরের Activity খোলে।
  - প্রথম বিকল্পে দ্বিতীয় Activity ব্যবহারকারীর কাছ থেকে একটি সাধারণ টেক্সট নেয়, আর তৃতীয় Activity-তে একটি custom broadcast receiver তৈরি হয়, যা সেই টেক্সট গ্রহণ করে।
  - দ্বিতীয় বিকল্পে দ্বিতীয় Activity ব্যাটারির শতাংশের broadcast গ্রহণ করে; তৃতীয় Activity-তে কিছুই করতে হয় না।
- **B. Image Scale:** ইন্টারনেট থেকে একটি ছবি লোড করে pinch gesture দিয়ে সেটিকে বড়-ছোট (scale) করা।
- **C. Video:** অ্যাপের ভেতরেই একটি ভিডিও চালানো (media player)।
- **D. Audio:** অ্যাপের ভেতরেই একটি অডিও চালানো (media player)।

১.২ অ্যাপের ফিচারসমূহ

তৈরি অ্যাপটি ওপরের প্রতিটি শর্ত পূরণ করে, আর বাস্তব অ্যাপের মতো আচরণের জন্য কিছু খুঁটিনাটিও যোগ করে। নিচের তালিকাটি পুরো নির্দেশিকার মানচিত্র; শেষ কলামের অধ্যায়ে প্রতিটি ফিচার বিস্তারিত ব্যাখ্যা করা হয়েছে।

ফিচার	কী করে	অধ্যায়
Navigation Drawer	চারটি স্ক্রিনের মধ্যে যাতায়াত; প্রতিটি স্ক্রিনে hamburger (☰) আইকন; toolbar-এর শিরোনাম ও drawer-এর হাইলাইট বর্তমান স্ক্রিন অনুসরণ করে; drawer খোলা থাকলে Back আগে drawer বন্ধ করে।	৬
Custom broadcast	দ্বিতীয় Activity-তে লেখা টেক্সট একটি custom broadcast হয়ে তৃতীয় Activity-র নিজস্ব receiver-এ পৌঁছায়। Receiver register আছে কি না, কী পেল, কখন পেল, সবই স্ক্রিনে দেখা যায়।	৭
Battery broadcast	সিস্টেমের `ACTION_BATTERY_CHANGED` broadcast থেকে ব্যাটারির শতাংশ, চার্জিং অবস্থা ও শেষ আপডেটের সময় লাইভ দেখায়; শুধু স্ক্রিন দৃশ্যমান থাকলেই শোনে।	৭

ফিচার	কী করে	অধ্যায়
Image Scale	ইন্টারনেট থেকে উচ্চ রেজোলিউশনের ছবি নামায়; দুই আঙুলে pinch zoom (১x থেকে ৫x), zoom অবস্থায় এক আঙুলে pan, double-tap-এ zoom বা রিসেট; loading ও ত্রুটির (Retry) অবস্থা।	৮
Video	অ্যাপে রাখা একটি ভিডিও VideoView ও MediaController দিয়ে চলে; rotation-এ থামে না; স্ক্রিন ছেড়ে গেলে থেমে একই জায়গায় অপেক্ষা করে।	৯
Audio	অ্যাপে রাখা একটি MP3 MediaPlayer দিয়ে চলে; Play/Pause, Stop, seek slider ও mm:ss সময়; rotation-এ গান চলতে থাকে, স্ক্রিন ছাড়লে থামে।	১০
সব স্ক্রিনে	Rotation-এ কোনো তথ্য হারায় না; landscape layout; light ও dark theme; edge-to-edge ডিজাইন।	১১

এই ফিচারগুলোর পেছনে দুটি তাত্ত্বিক ধারণা বারবার কাজ করে: **Message Queue** ও **Publish/Subscribe**। তাই ফিচারের আগে অধ্যায় ২ ও ৩-এ এই দুটি ধারণা উদাহরণসহ বিস্তারিত ব্যাখ্যা করা হয়েছে।

১.৩ ব্যবহৃত প্রযুক্তি

- **ভাষা ও UI:** Kotlin; স্ক্রিনগুলো XML layout দিয়ে তৈরি এবং ViewBinding দিয়ে কোডের সঙ্গে যুক্ত। Jetpack Compose ব্যবহার করা হয়নি।
- **Material 3:** MaterialToolbar, NavigationView, TextInputLayout, Slider, progress indicator ইত্যাদি component; light ও dark দুই theme।
- **Jetpack (AndroidX):** Navigation (navigation graph, NavHostFragment, NavigationUI), Fragment, Lifecycle, ViewModel, SavedStateHandle ও LiveData; সঙ্গে Kotlin coroutine ও StateFlow।
- **Coil 3:** ইন্টারনেট থেকে ছবি নামানো ও decode করার লাইব্রেরি (নেটওয়ার্কের জন্য OkHttp)।
- **Android-এর media API:** `android.media.MediaPlayer`, VideoView ও MediaController; বাড়তি কোনো player লাইব্রেরি নেই।
- **SDK:** minSdk 24 (Android 7.0), targetSdk ও compileSdk 37।

১.৪ কোডের মানচিত্র ও "কোডে কোথায়" বাক্স

সব Kotlin ফাইল আছে `app/src/main/java/com/example/labassignment2_cse489/` ফোল্ডারে, আর layout, menu, navigation ও string-এর মতো resource ফাইল আছে `app/src/main/res/`-এ। নিচের মানচিত্রে পথগুলো এই ফোল্ডারের পর থেকে লেখা।

ফাইল	কী আছে	অধ্যায়
`MainActivity.kt`	একমাত্র host Activity: drawer, toolbar, navigation, Back ও insets	৬
`broadcast/BroadcastFragment.kt`	Broadcast-এর প্রথম স্ক্রিন: Spinner ও Proceed	৭
`broadcast/BroadcastViewModel.kt`	Spinner-এর নির্বাচন মনে রাখা	৭
`broadcast/BroadcastMode.kt`	দুটি mode: CUSTOM ও BATTERY	৭

ফাইল	কী আছে	অধ্যায়
`broadcast/BroadcastInputActivity.kt`	Activity 2: টেক্সট নেওয়া, অথবা লাইভ ব্যাটারি দেখানো	৭
`broadcast/CustomReceiverActivity.kt`	Activity 3: receiver register, broadcast পাঠানো ও গ্রহণ	৭
`broadcast/CustomMessageReceiver.kt`	নিজস্ব BroadcastReceiver, অর্থাৎ subscriber	৭
`broadcast/CustomReceiverViewModel.kt`	প্রাপ্ত বার্তা ও "পাঠানো হয়েছে" flag	৭
`broadcast/BatteryStatusLiveData.kt`	দৃশ্যমান থাকলেই ব্যাটারির broadcast শোনে	৭
`broadcast/BatteryViewModel.kt`	ব্যাটারির LiveData ধরে রাখে	৭
`broadcast/BatteryStatus.kt`	শতাংশের হিসাব ও চার্জিং অবস্থা	৭
`image/ImageScaleFragment.kt`	Image Scale স্ক্রিন: loading, ত্রুটি, zoom লেবেল	৮
`image/ImageScaleViewModel.kt`	Coil দিয়ে ছবি নামানো	৮
`image/ZoomableImageView.kt`	Pinch, pan, double-tap ও Matrix	৮
`image/ZoomMath.kt`	Zoom-এর বিশুদ্ধ গণিত	৮
`video/VideoFragment.kt`	VideoView ও MediaController	৯
`video/VideoViewModel.kt`	ভিডিওর অবস্থান মনে রাখা	৯
`audio/AudioFragment.kt`	অডিওর বাটন, slider ও সময়	১০
`audio/AudioPlayerViewModel.kt`	MediaPlayer-এর মালিক	১০
`util/TimeFormat.kt`, `util/WindowInsetsExt.kt`	mm:ss ও ঘড়ির সময়; সিস্টেম বারের জন্য padding	১০, ৬
`AndroidManifest.xml`, `res/navigation/nav_graph.xml`, `res/menu/drawer_menu.xml`	Permission ও Activity-র তালিকা; চারটি স্ক্রিনের graph; drawer-এর item	৬

কোড ব্যাখ্যা করা প্রতিটি অংশের শুরুতে একটি **"কোডে কোথায়"** বাক্স আছে। এর প্রতিটি চিপ তিনটি তথ্য দেয়: ফাইলের নাম, function বা ভেরিয়েবলের নাম, এবং লাইন নম্বর। যেমন `CustomMessageReceiver.kt › onReceive() লাইন 23–28` মানে `CustomMessageReceiver.kt` ফাইলের `onReceive()` function, লাইন 23 থেকে 28। লেখার ভেতরে শুধু লাইন নম্বরের ছোট চিপও আছে, যেমন `লাইন 34`; তখন কোন ফাইল, তা আশপাশের লেখা বা বাক্স থেকে বোঝা যায়।

> **কোড মিলিয়ে পড়ার নিয়ম**
>
> VS Code বা Android Studio-তে ফাইলটি খুলে Ctrl+G চেপে লাইন নম্বর দিলে সরাসরি সেখানে যাওয়া যায়। লাইন নম্বরগুলো কোডের বর্তমান সংস্করণ (git commit `50afc1c`) অনুযায়ী; পরে কোড বদলালে নম্বর একটু সরে যেতে পারে, তখন function-এর নাম দিয়ে খুঁজলেই হবে।

২ Message Queue: ধারণা ও উদাহরণ

এই অধ্যায়ে message queue বা বার্তার সারির ধারণাটি একদম গোড়া থেকে দেখব: কেন দরকার, কীভাবে কাজ করে, দৈনন্দিন জীবনে ও সফটওয়্যারে এর উদাহরণ, এবং শেষে এই অ্যাপের ভেতরে এটি কোথায় কাজ করছে।

২.১ সরাসরি ডাকার সমস্যা

ধরা যাক একটি প্রোগ্রামের অংশ A-কে দিয়ে অংশ B-কে একটি কাজ করাতে হবে। সবচেয়ে সহজ উপায় হলো A সরাসরি B-এর function ডাকবে। একে **synchronous call** বা সরাসরি ডাকা বলে। এতে চার রকম সমস্যা হতে পারে:

- **অপেক্ষা:** B কাজ শেষ না করা পর্যন্ত A আটকে থাকে। B ধীর হলে A-ও ধীর হয়ে যায়।
- **উপস্থিতি:** ডাকার মুহূর্তে B-কে চালু ও প্রস্তুত থাকতে হয়। B বন্ধ থাকলে কাজটি ব্যর্থ হয়।
- **চাপ:** একসঙ্গে অনেক অনুরোধ এলে B সামলাতে পারে না, আর অনুরোধগুলো জমিয়ে রাখার কোনো জায়গাও নেই।
- **শক্ত বাঁধন (tight coupling):** A-কে জানতে হয় B কে, কোথায় আছে, কীভাবে ডাকতে হয়। B বদলালে A-কেও বদলাতে হয়।

দৈনন্দিন জীবনে এটি ফোন কলের মতো: অপর পক্ষ ফোন না ধরলে কথা হয় না, আর কথা শেষ না হওয়া পর্যন্ত দুজনেই ব্যস্ত। এর বিকল্প হলো SMS বা চিঠি: প্রেরক বার্তা পাঠিয়েই নিজের কাজে ফিরে যান, আর প্রাপক সুবিধামতো সময়ে পড়ে নেন। সফটওয়্যারে এই "বার্তা পাঠানো"-র দুটি প্রধান রূপ হলো message queue ও publish/subscribe। এই অধ্যায়ে প্রথমটি, পরের অধ্যায়ে দ্বিতীয়টি।

২.২ Message Queue কী

Message queue হলো একটি সারি (queue), যেখানে এক পক্ষ বার্তা রেখে যায় আর অন্য পক্ষ সেখান থেকে একটি একটি করে বার্তা তুলে নিয়ে কাজ করে। এর চারটি অংশ:

- **Message (বার্তা):** যে কাজ করতে হবে বা যে খবর দিতে হবে, তার একটি প্যাকেট। সাধারণত দুই ভাগে থাকে: header (কী ধরনের বার্তা, কখন তৈরি) এবং body বা payload (আসল ডেটা)।
- **Producer (প্রেরক):** যে বার্তা তৈরি করে queue-তে রাখে। Queue-তে রাখার এই কাজকে enqueue বলে।
- **Queue (সারি):** বার্তাগুলো যেখানে অপেক্ষা করে। এটি প্রেরক ও গ্রাহকের মাঝখানের একটি বাফার।
- **Consumer (গ্রাহক):** যে queue থেকে বার্তা তুলে নেয় (dequeue) এবং প্রক্রিয়া করে।

Queue সাধারণত FIFO (First In, First Out) নিয়মে চলে: যে বার্তা আগে এসেছে, সেটি আগে বের হয়। ধরা যাক producer পরপর তিনটি বার্তা পাঠাল: M1, M2, M3। তখন queue-এর অবস্থা এরকম:

```
Producer → [ M3 | M2 | M1 ] → Consumer
```

Consumer প্রথমে M1 নেবে, তারপর M2, তারপর M3। Consumer যতক্ষণ M1 নিয়ে ব্যস্ত, ততক্ষণ M2 ও M3 সারিতে অপেক্ষা করে; হারিয়ে যায় না। এর মধ্যে producer চাইলে M4, M5 পাঠাতে পারে; সেগুলো সারির

পেছনে যোগ হয়।

> **মূল নিয়ম**
> Message queue-তে প্রতিটি বার্তা ঠিক একজন consumer প্রক্রিয়া করে, এবং প্রক্রিয়া শেষ হলে বার্তাটি queue থেকে মুছে যায়। একে **point-to-point** যোগাযোগ বলে: একটি বার্তা, একজন প্রাপক।

২.৩ দৈনন্দিন উদাহরণ: রেস্তোরাঁ ও ব্যাংক

উদাহরণ ১: রেস্তোরাঁর রান্নাঘর

একটি ব্যস্ত রেস্তোরাঁয় waiter টেবিল থেকে অর্ডার নিয়ে একটি স্লিপে লেখেন এবং রান্নাঘরের তারে স্লিপটি ঝুলিয়ে দেন। রাঁধুনি সেই তার থেকে সবচেয়ে পুরোনো স্লিপটি নেন, রান্না করেন, তারপর পরের স্লিপ। এখানে প্রতিটি অংশ message queue-এর একটি অংশের সঙ্গে মেলে:

রেস্তোরাঁয়	Message queue-তে
অর্ডারের স্লিপ	Message
Waiter	Producer
রান্নাঘরের তার, যেখানে স্লিপ ঝোলে	Queue
রাঁধুনি	Consumer

এবার দেখা যাক এই ব্যবস্থায় কী কী ঘটে, আর প্রতিটি ঘটনার নাম কী:

১. Waiter স্লিপ ঝুলিয়েই পরের টেবিলে চলে যান; রান্না শেষ হওয়ার অপেক্ষা করেন না। **প্রেরক অপেক্ষা করে না (asynchronous)।**
২. দুপুরের ভিড়ে একসঙ্গে দশটি অর্ডার এলেও কোনোটি হারায় না; স্লিপগুলো তারে জমে থাকে, আর রাঁধুনি নিজের গতিতে একটি একটি করে রান্না করেন। **বাফারিং বা load leveling।**
৩. যে অর্ডার আগে এসেছে, সেটি আগে রান্না হয়। **FIFO ক্রম।**
৪. দুজন রাঁধুনি থাকলে তাঁরা স্লিপগুলো ভাগ করে নেন; একই স্লিপ দুজনে রান্না করেন না। **Competing consumers: প্রতিটি বার্তা একজনই প্রক্রিয়া করে।**
৫. রান্না শেষ হলে স্লিপটি সেই তার থেকে নামিয়ে ফেলা হয়। **প্রক্রিয়ার পর বার্তা queue থেকে মুছে যায়।**
৬. একজন রাঁধুনি একটি অর্ডারে আধা ঘণ্টা আটকে থাকলে পেছনের সব অর্ডার দেরি হয়ে যায়। **একজন consumer থাকলে একটি ধীর বার্তা পুরো সারিকে আটকে দেয়।** এই শেষ বিষয়টি Android-এর জন্য খুব গুরুত্বপূর্ণ (২.৬)।

উদাহরণ ২: ব্যাংকের টোকেন

ব্যাংকে ঢুকে গ্রাহক মেশিন থেকে একটি টোকেন নেন; অর্থাৎ বার্তা queue-তে ঢুকল। কাউন্টারগুলো (consumer) একে একে পরের নম্বর ডাকে। গ্রাহক বসে অপেক্ষা করেন, লাইন ভাঙে না, আর প্রতিটি টোকেন একটিমাত্র কাউন্টারে সেবা পায়। কাউন্টার বাড়লে সারি দ্রুত ছোট হয়, কিন্তু নিয়ম একই থাকে: এক টোকেন, এক কাউন্টার।

২.৪ সফটওয়্যারের উদাহরণ: অনলাইন দোকান

একটি অনলাইন দোকানের কথা ভাবা যাক। গ্রাহক "Order" চাপলে সার্ভারকে তিনটি কাজ করতে হয়: অর্ডার সংরক্ষণ, নিশ্চিতকরণ ইমেইল পাঠানো এবং গুদামে প্যাকিংয়ের নির্দেশ দেওয়া। ইমেইল পাঠাতে কয়েক সেকেন্ড লাগতে পারে, আর ইমেইল সার্ভার মাঝে মাঝে বন্ধও থাকে।

- **Queue ছাড়া:** সার্ভার ইমেইল পাঠানো শেষ না হওয়া পর্যন্ত গ্রাহককে অপেক্ষা করায়। ইমেইল সার্ভার বন্ধ থাকলে পুরো অর্ডারই ব্যর্থ হতে পারে, যদিও অর্ডারের আসল কাজে কোনো সমস্যা ছিল না।
- **Queue দিয়ে:** সার্ভার অর্ডার সংরক্ষণ করে "OrderPlaced #1023" নামের একটি বার্তা email queue-তে রাখে, আর সঙ্গে সঙ্গে গ্রাহককে জানায় "অর্ডার হয়েছে"। আলাদা একটি email worker (consumer) queue থেকে বার্তা নিয়ে ইমেইল পাঠায়। ইমেইল সার্ভার বন্ধ থাকলে বার্তাগুলো queue-তে অপেক্ষা করে; চালু হলে worker সেগুলো একে একে পাঠিয়ে দেয়।

ফলাফল: কোনো অর্ডারের ইমেইল হারায় না, গ্রাহককেও অপেক্ষা করতে হয় না, আর ঈদের সেলের মতো ভিড়ের সময় আরও কয়েকটি email worker চালু করলেই কাজ ভাগ হয়ে যায়। বাস্তবে এ কাজে RabbitMQ, Amazon SQS বা ActiveMQ-এর মতো সফটওয়্যার ব্যবহার হয়।

২.৫ Message queue-এর গুরুত্বপূর্ণ বৈশিষ্ট্য

- **ক্রম (ordering):** সাধারণ queue FIFO। কিছু queue বার্তাকে অগ্রাধিকার (priority) বা নির্ধারিত সময় অনুযায়ী সাজায়; তখন জরুরি বার্তা, কিংবা যার নির্ধারিত সময় আগে, সেটি আগে বের হয়।
- **বাফারিং ও load leveling:** producer দ্রুত আর consumer ধীর হলে বার্তা queue-তে জমে; consumer নিজের গতিতে কাজ করে। ভিড়ের সময়ের চাপ সময়ের ওপর ছড়িয়ে যায়।
- **প্রতিটি বার্তা একজন consumer:** একাধিক consumer থাকলে তারা কাজ ভাগ করে নেয় (competing consumers)। consumer বাড়িয়ে কাজের গতি বাড়ানো যায়।
- **Acknowledgement (ack) ও retry:** consumer কাজ শেষ করে queue-কে জানায় "শেষ হয়েছে", তবেই বার্তা মোছা হয়। Ack আসার আগে consumer বিকল হলে queue বার্তাটি আরেকজনকে আবার দেয়।
- **Delivery-র নিশ্চয়তা:** at-most-once মানে বার্তা হারাতে পারে কিন্তু দুবার আসবে না; at-least-once মানে হারাবে না কিন্তু দুবার আসতে পারে, তাই গ্রাহককে duplicate সামলাতে হয়; exactly-once মানে ঠিক একবার, যা অর্জন করা সবচেয়ে কঠিন।
- **দেরিতে চলার বার্তা (delayed message):** বার্তা এখনই নয়, নির্দিষ্ট সময় পরে প্রক্রিয়ার জন্য রাখা যায়; যেমন "৩০ মিনিট পরে মনে করিয়ে দাও"।
- **Durability:** queue যদি বার্তা disk-এ লিখে রাখে, তবে সার্ভার বন্ধ-চালু হলেও বার্তা থাকে। শুধু মেমরিতে থাকলে (in-memory queue) প্রসেস বন্ধ হলে অপেক্ষমাণ বার্তা হারায়।

সুবিধা	সীমাবদ্ধতা
প্রেরক অপেক্ষা করে না; সাড়া দ্রুত হয়	ফল সঙ্গে সঙ্গে পাওয়া যায় না (কিছুটা দেরি, বা latency)
ভিড় সামলায়; কোনো বার্তা হারায় না	Queue নিজেই একটি বাড়তি অংশ, যা চালাতে ও দেখভাল করতে হয়
প্রেরক ও গ্রাহক আলাদাভাবে বদলানো যায়	একাধিক consumer থাকলে কঠোর ক্রম বজায় রাখা কঠিন

২.৬ Android-এর main thread: অ্যাপের ভেতরের message queue

কোডে কোথায় এই অংশ Android-এর নিজস্ব ব্যবস্থা; প্রজেক্টে এটি সরাসরি লেখা নেই। তবে নিচের **২.৭** অংশে এর প্রতিটি ব্যবহার কোডের অবস্থানসহ দেখানো হয়েছে।

Android নিজেই প্রতিটি অ্যাপের ভেতরে একটি message queue চালায়। অ্যাপের সব UI কাজ একটিমাত্র thread-এ হয়: **main thread** (UI thread)। এই thread-এ চারটি class মিলে একটি পূর্ণ message queue ব্যবস্থা তৈরি করে:

সাধারণ ধারণা	Android class	কাজ
Message	Message	একটি কাজ (বা চালানোর জন্য একটি Runnable), সঙ্গে সেটি কখন চলবে তার সময়
Queue	MessageQueue	বার্তাগুলো চলার সময় অনুযায়ী সাজিয়ে রাখে; দেরিতে চলার বার্তা তার সময় পর্যন্ত অপেক্ষা করে
Producer	Handler	`post()` বা `sendMessageDelayed()` দিয়ে queue-তে বার্তা রাখে; সিস্টেম নিজেও বার্তা রাখে
Consumer	Looper	একটি অনন্ত লুপ: পরের বার্তা তুলে নিয়ে চালায়, তারপর আবার পরেরটা

এই queue-র consumer একজনই, Looper। তাই বার্তাগুলো একটির পর একটি চলে, দুটি কখনো একসঙ্গে নয়। এর দুটি গুরুত্বপূর্ণ ফল:

- **Lock লাগে না:** একসঙ্গে দুটি callback কখনো চলে না, তাই UI-এর ডেটা পড়তে-লিখতে lock বা বাড়তি সাবধানতা লাগে না।
- **ধীর বার্তা সব আটকে দেয়:** কোনো বার্তা প্রক্রিয়া করতে বেশি সময় লাগলে পেছনের সব বার্তা অপেক্ষা করে; touch সাড়া দেয় না, স্ক্রিন জমে যায়, আর কয়েক সেকেন্ড চললে সিস্টেম **ANR** (Application Not Responding) দেখায়। এটি সেই আধা ঘণ্টা আটকে থাকা রাঁধুনির ঘটনা। তাই নেটওয়ার্ক বা ফাইল প্রস্তুত করার মতো ধীর কাজ main thread-এ করা চলে না।

আরেকটি বিষয়: এই queue in-memory, অর্থাৎ অ্যাপের প্রসেস বন্ধ হলে অপেক্ষমাণ বার্তাগুলোও হারায়। এ কারণেই দরকারি তথ্য আলাদাভাবে সংরক্ষণ করতে হয় (**৫.২**)।

২.৭ এই অ্যাপে message queue-এর উদাহরণ

এই অ্যাপের প্রায় প্রতিটি callback আসলে main thread-এর queue থেকে আসা একটি বার্তা। কয়েকটি স্পষ্ট উদাহরণ, কোডের অবস্থানসহ:

১. **Touch event:** আঙুলের প্রতিটি নড়াচড়া একটি MotionEvent বার্তা হয়ে queue-তে আসে, আর Looper সেটি view-এর `onTouchEvent()`-এ পৌঁছে দেয়। Image Scale স্ক্রিন এই বার্তার ধারা থেকেই pinch ও pan চেনে।

`ZoomableImageView.kt` › `onTouchEvent()` লাইন 170–187

২. **Broadcast-এর `onReceive()`:** সিস্টেম থেকে আসা broadcast প্রথমে আমাদের প্রসেসের main queue-তে জমা হয়; Looper পৌঁছালে `onReceive()` চলে। তাই সেখান থেকে সরাসরি LiveData-র মান বসানো

নিরাপদ, কোডের মন্তব্যেও সে কথা লেখা আছে।

`BatteryStatusLiveData.kt › receiver` লাইন 22–27

৩. **MediaPlayer-এর onPrepared:** `prepareAsync()` অনুরোধ পাঠিয়েই ফিরে আসে; ফাইল প্রস্তুত হলে player main queue-তে একটি "প্রস্তুত" বার্তা রাখে, আর তখন onPrepared চলে। এটি asynchronous request/reply: অনুরোধ এখন, উত্তর পরে বার্তা হিসেবে।

`AudioPlayerViewModel.kt › init (listener & prepareAsync)` লাইন 64–69

৪. **Delayed বার্তা, `delay(200)`:** অডিওর progress লুপ প্রতি ২০০ মিলিসেকেন্ডে অবস্থান হালনাগাদ করে। `delay()` আসলে একটি delayed বার্তা রেখে coroutine-কে থামিয়ে দেয়; ২০০ মিলিসেকেন্ড পরে বার্তাটি চলার উপযোগী হলে লুপ আবার চলে, আর মাঝের সময়ে main thread অন্য বার্তা প্রক্রিয়া করে।

`AudioPlayerViewModel.kt › startProgressUpdates()` লাইন 125–133

৫. **Animation-এর frame:** double-tap-এর zoom animation প্রতিটি frame-এ একটি update callback পায়, যা একটি করে বার্তা হিসেবে আসে।

`ZoomableImageView.kt › animateZoomTo()` লাইন 158–168

৬. **Lifecycle callback:** onStart, onStop ইত্যাদিও সিস্টেম থেকে আসা বার্তা হিসেবে main thread-এ চলে; যেমন Activity 3-এর receiver register হয় onStart-এ।

`CustomReceiverActivity.kt › onStart()` লাইন 79–103

২.৮ একটি বার্তার যাত্রা: Play চাপা থেকে slider চলা

কোডে কোথায় `AudioFragment.kt › onViewCreated()` লাইন 30–44
`AudioPlayerViewModel.kt › togglePlayPause(), play()` লাইন 72–81
`AudioPlayerViewModel.kt › startProgressUpdates()` লাইন 125–133
`AudioFragment.kt › render()` লাইন 76–103

Audio স্ক্রিনে ব্যবহারকারী Play চাপলে ভেতরে কোন কোন বার্তা চলাচল করে, ধাপে ধাপে দেখা যাক:

১. আঙুল স্ক্রিন ছোঁয়া ও তোলার সঙ্গে সঙ্গে দুটি touch বার্তা (ACTION_DOWN ও ACTION_UP) main queue-তে আসে। Looper এগুলো Play বাটনে পৌঁছে দেয়, আর বাটন বুঝে নেয় এটি একটি click।

২. Click listener ViewModel-এর `togglePlayPause()` ডাকে লাইন 34 ।

৩. ViewModel দেখে গান চলছে না, তাই `play()` চালায়: player-কে start করে, state-এ "চলছে" লেখে, এবং progress লুপ চালু করে লাইন 76–81 ।

৪. State বদলানোয় StateFlow নতুন মান প্রকাশ করে। Fragment-এর collector, যা নিজেও main thread-এর একটি বার্তা হিসেবে চলে, `render()` চালায়, আর বাটনের লেখা "Pause" হয়ে যায়।

৫. Progress লুপ player-এর বর্তমান অবস্থান পড়ে state-এ বসায়, তারপর `delay(200)` । ২০০ মিলিসেকেন্ড পরে delayed বার্তাটি চলার উপযোগী হয়, Looper লুপটিকে আবার চালায়, আর slider এক ধাপ এগোয়। গান চলার পুরো সময় এটি চলতে থাকে।

৬. এই সবের ফাঁকে ফাঁকে ব্যবহারকারীর অন্য touch, স্ক্রিন আঁকার frame ইত্যাদি বার্তাও একই queue দিয়ে চলে। কোনো বার্তাই বেশি সময় নেয় না, তাই অ্যাপ মসৃণ থাকে।

মনে রাখার বিষয়

Main thread একটি message queue, যার consumer একজন: Looper। অ্যাপের প্রায় সব কোড কোনো না কোনো বার্তার উত্তরে চলে, তাই প্রতিটি callback দ্রুত শেষ করা জরুরি। ধীর কাজ (ডাউনলোড, ফাইল প্রস্তুত) অন্য thread-এ পাঠিয়ে ফল আবার বার্তা হিসেবে main thread-এ আনতে হয়, ঠিক যেমন Coil ও `prepareAsync()` করে।

৩ Publish/Subscribe: ধারণা ও উদাহরণ

Message queue-তে একটি বার্তা যায় একজন প্রাপকের কাছে। কিন্তু অনেক সময় একটি খবর একসঙ্গে অনেকের জানা দরকার, আর প্রেরক জানেনও না কারা জানতে চায়। এর সমাধান publish/subscribe। এই অধ্যায়ে এর ধারণা, উদাহরণ এবং Android ও এই অ্যাপে এর প্রয়োগ দেখব।

৩.১ Pub/Sub কী

Publish/Subscribe (সংক্ষেপে Pub/Sub) এমন একটি বার্তা ব্যবস্থা, যেখানে প্রেরক নির্দিষ্ট কাউকে বার্তা পাঠায় না; একটি **topic** (বিষয়)-এ বার্তা প্রকাশ (publish) করে। যারা আগে থেকে ওই topic-এ আগ্রহ জানিয়ে রেখেছে (subscribe), তাদের প্রত্যেকে বার্তার একটি করে কপি পায়। এর অংশগুলো:

- **Publisher (প্রকাশক):** যে বার্তা প্রকাশ করে। সে জানে না কে বা কতজন শুনছে।
- **Subscriber (গ্রাহক):** যে এক বা একাধিক topic-এ subscribe করে এবং সেসব topic-এর বার্তা পায়।
- **Topic (বিষয়):** বার্তার শ্রেণির নাম, যেমন "খেলার খবর" বা "ব্যাটারির অবস্থা"। Publisher ও subscriber শুধু topic-এর নামে একমত হয়।
- **Broker (মধ্যস্থতাকারী):** মাঝখানের ব্যবস্থা, যে কে কোন topic-এ subscribe করেছে তার তালিকা রাখে এবং প্রতিটি বার্তা মিলিয়ে সঠিক subscriber-দের কাছে পৌঁছে দেয়।
- **Subscription:** "এই subscriber এই topic চায়", এই নিবন্ধন। Unsubscribe করলে নিবন্ধন মুছে যায়, আর বার্তা আসা বন্ধ হয়।

> **মূল নিয়ম**
> Message queue-তে একটি বার্তা যায় একজনের কাছে; pub/sub-এ একটি বার্তার কপি যায় topic-এর সব subscriber-এর কাছে। একে **fan-out** বলে। আর publisher-কে subscriber-দের চিনতে হয় না, শুধু topic-এর নাম জানলেই চলে।

৩.২ দৈনন্দিন উদাহরণ: YouTube, রেডিও ও নোটিশ বোর্ড

উদাহরণ ১: YouTube চ্যানেল

কেউ একটি YouTube চ্যানেলে Subscribe চাপলে তিনি ওই চ্যানেলের subscriber হয়ে যান। চ্যানেল নতুন ভিডিও আপলোড করলে (publish) YouTube (broker) চ্যানেলের সব subscriber-কে notification পাঠায়। চ্যানেলের মালিক জানেন না ঠিক কোন কোন মানুষ notification পেলেন; তিনি শুধু আপলোড করেন। নতুন subscriber এলে বা কেউ unsubscribe করলে মালিককে কিছুই বদলাতে হয় না।

YouTube-এ	Pub/Sub-এ
চ্যানেল	Topic
ভিডিও আপলোড করা	Publish
Subscribe বাটন চাপা	Subscribe
YouTube-এর notification ব্যবস্থা	Broker

YouTube-এ	Pub/Sub-এ
দর্শক	Subscriber
প্রত্যেকের ফোনে আসা notification	প্রতিটি subscriber-এর কাছে পৌঁছানো বার্তার কপি

উদাহরণ ২: রেডিও

রেডিও স্টেশন একটি নির্দিষ্ট frequency-তে (topic) অনুষ্ঠান প্রচার করে। যার রেডিও সেই frequency-তে টিউন করা (subscribe), তিনিই শোনেন। কিন্তু কেউ দেরিতে রেডিও চালু করলে আগের খবর আর শুনতে পান না; যা প্রচার হয়ে গেছে, তা চলে গেছে। একে বলে **non-durable** delivery: শুধু তখন উপস্থিত গ্রাহকই বার্তা পান।

উদাহরণ ৩: স্কুলের নোটিশ বোর্ড

নোটিশ বোর্ডে সর্বশেষ নোটিশটি ঝুলে থাকে, যতক্ষণ না নতুন নোটিশ এসে তার জায়গা নেয়। যে ছাত্র পরে আসে, সে-ও বোর্ড দেখেই সর্বশেষ খবর জেনে যায়। একে বলে **retained** বা last-value বার্তা: broker প্রতিটি topic-এর শেষ বার্তাটি রেখে দেয়, আর নতুন subscriber যুক্ত হওয়ামাত্র সেটি পায়।

পরে দেখব, এই অ্যাপের custom broadcast রেডিওর মতো (**৩.৯**), আর ব্যাটারির broadcast নোটিশ বোর্ডের মতো (**৩.১০**)।

৩.৩ ধাপে ধাপে: একটি ক্রিকেট স্কোর অ্যাপ

একটি ক্রিকেট স্কোর অ্যাপের উদাহরণ নেওয়া যাক। সার্ভারে একটি broker আছে, আর topic-এর নাম `score/bd-vs-ind`।

১. **Subscribe:** রহিমের ফোন ও করিমের ফোন broker-কে জানায়, "আমরা score/bd-vs-ind চাই"। Broker তালিকায় লিখে রাখে: score/bd-vs-ind → রহিম, করিম।

২. **Publish:** মাঠের স্কোরার একটি চার হওয়ার পর বার্তা পাঠান: topic score/bd-vs-ind, বিষয়বস্তু "বাংলাদেশ ১৫০/৩"। তিনি কারও নাম লেখেন না, পাঠিয়েই পরের বলে মন দেন।

৩. **Match:** Broker বার্তার topic দেখে তালিকা থেকে subscriber খুঁজে পায়: রহিম ও করিম।

৪. **Fan-out ও deliver:** Broker দুজনের প্রত্যেককে বার্তার একটি করে কপি পাঠায়।

৫. **Consume:** প্রতিটি ফোন নিজের মতো বার্তা প্রক্রিয়া করে: একটিতে notification দেখায়, অন্যটিতে স্কোরবোর্ড হালনাগাদ করে।

৬. **Unsubscribe:** খেলা শেষে করিম অ্যাপ বন্ধ করলে তাঁর ফোন unsubscribe করে; এরপর বার্তা শুধু রহিমের কাছে যায়। স্কোরারকে এ ব্যাপারে কিছুই জানতে বা করতে হয় না।

বাস্তবে এ ধরনের ব্যবস্থায় MQTT, Redis Pub/Sub, Apache Kafka বা Google Cloud Pub/Sub ব্যবহার করা হয়। ফোনের push notification-ও (যেমন Firebase Cloud Messaging-এর topic) একই নীতিতে চলে।

৩.৪ Topic ও filter

Subscriber সব বার্তা চায় না, নির্দিষ্ট কিছু চায়। কোন বার্তা কোন subscriber পাবে, তা দুইভাবে ঠিক হয়:

- **Topic-ভিত্তিক:** বার্তার গায়ে একটি topic-এর নাম থাকে; subscriber যে topic-এ subscribe করেছে, শুধু সেটির বার্তা পায়। ওপরের ক্রিকেট উদাহরণ এরকম।

- **বিষয়বস্তু-ভিত্তিক (content-based):** subscriber একটি শর্ত দেয়, যেমন "শুধু উইকেট পড়লে জানাও"; broker বার্তার ভেতরের তথ্য দেখে শর্ত মেলায়।

Android-এ এই filter-এর নাম **IntentFilter**। এটি মূলত topic-ভিত্তিক (Intent-এর action মেলায়), তবে category ও data দিয়েও শর্ত দেওয়া যায়। এই অ্যাপের দুটি filter-ই শুধু একটি করে action মেলায়:

কোডে কোথায় `CustomMessageReceiver.kt` › `intentFilter()` লাইন 40
`BatteryStatusLiveData.kt` › `onActive() (ব্যাটারির filter)` লাইন 37–38

৩.৫ তিন ধরনের decoupling

Pub/Sub প্রেরক ও প্রাপককে তিন দিক থেকে আলাদা করে:

ধরন	মানে	উদাহরণ
Space decoupling	Publisher ও subscriber একে অপরকে চেনে না; শুধু topic-এর নাম জানে।	YouTube চ্যানেল তার দর্শকদের নাম জানে না।
Synchronization decoupling	Publisher বার্তা পাঠিয়েই নিজের কাজে ফেরে; কারও গ্রহণের অপেক্ষা করে না।	স্কোরার পাঠিয়েই পরের বলে মন দেন; Android-এ `sendBroadcast()` সঙ্গে সঙ্গে ফিরে আসে।
Time decoupling	দুই পক্ষের একই সময়ে সক্রিয় থাকার দরকার নেই; broker বার্তা ধরে রাখতে পারে।	নোটিশ বোর্ড; Android-এর sticky broadcast।

খেয়াল করার বিষয়, রেডিও-ধরনের সাধারণ pub/sub-এ time decoupling নেই: subscriber-কে পাঠানোর মুহূর্তে উপস্থিত থাকতে হয়। Android-এর সাধারণ broadcast-ও এমন, আর এটিই এই অ্যাপের একটি গুরুত্বপূর্ণ নকশার কারণ (**৭.৭**)।

৩.৬ দেরিতে আসা subscriber: তিন রকম delivery

Pub/Sub-এর সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন: বার্তা প্রকাশের সময় যে subscriber ছিল না, সে কি পরে বার্তাটি পাবে? তিন রকম উত্তর হতে পারে:

ধরন	কী হয়	দৈনন্দিন উদাহরণ	Android-এ
Non-durable	শুধু উপস্থিত subscriber পায়; দেরিতে আসা কেউ পায় না।	রেডিও	সাধারণ broadcast (`sendBroadcast()`)
Retained (last value)	Broker শেষ বার্তাটি রেখে দেয়; নতুন subscriber সঙ্গে সঙ্গে সেটি পায়।	নোটিশ বোর্ড	Sticky broadcast (`ACTION_BATTERY_CHANGED`); LiveData ও StateFlow
Durable subscription	Subscriber না থাকার সময়ের সব বার্তা জমা থাকে; ফিরে এলে সব পায়।	ইমেইলের inbox	এই অ্যাপে ব্যবহৃত হয়নি

এই পার্থক্য না বুঝলে বার্তা "নিঃশব্দে হারিয়ে যাওয়া"-র মতো বাগ হয়: প্রেরক ভাবে পাঠানো হয়ে গেছে, কিন্তু গ্রাহক তখনও প্রস্তুত ছিল না। কোনো error-ও দেখা যায় না, কারণ non-durable ব্যবস্থায় "কেউ শুনছে না" কোনো ভুল নয়।

৩.৭ Observer pattern: একই প্রোগ্রামের ভেতরে pub/sub

Pub/Sub-এর ছোট, broker-ছাড়া রূপ হলো **Observer pattern**। এখানে একটি object (subject বা observable) নিজেই তার observer-দের তালিকা রাখে, আর নিজের মান বদলালে সবাইকে জানায়। সবকিছু একই প্রসেসের ভেতরে ঘটে, তাই আলাদা broker লাগে না।

Android Jetpack-এর LiveData ও StateFlow এই pattern-এর উদাহরণ, আর দুটিই retained বার্তার মতো আচরণ করে: নতুন observer যুক্ত হওয়ামাত্র বর্তমান মানটি পায়, তারপর প্রতিটি পরিবর্তন। এই অ্যাপে:

- Activity 3 প্রাপ্ত বার্তার LiveData observe করে; ViewModel নতুন বার্তা রাখলেই স্ক্রিন হালনাগাদ হয় লাইন 76।
- Audio স্ক্রিন একটি StateFlow collect করে; ViewModel state বদলালেই `render()` চলে লাইন 39–43।
- `ZoomableImageView`-এর `onZoomChanged` callback একজন মাত্র observer নিয়ে observer pattern-এর সবচেয়ে ছোট রূপ; zoom বদলালেই লেবেল হালনাগাদ হয় লাইন 36।

LiveData-র একটি বাড়তি গুণ আছে: এটি observer-এর lifecycle জানে, তাই স্ক্রিন দৃশ্যমান থাকলেই কেবল মান পৌঁছায়। ফলে subscription-এর মেয়াদ নিজে থেকেই স্ক্রিনের সঙ্গে মেলে (**৫.১**)।

৩.৮ Android-এর broadcast: একটি pub/sub ব্যবস্থা

Android-এর broadcast ব্যবস্থা হুবহু broker-ভিত্তিক pub/sub। কোনো অ্যাপ যখন `registerReceiver()` বা `sendBroadcast()` ডাকে, তখন সে receiver-এর সঙ্গে সরাসরি কথা বলে না; Binder (Android-এ এক প্রসেস থেকে আরেক প্রসেসে কথা বলার ব্যবস্থা) দিয়ে সিস্টেমের `system_server` প্রসেসের ActivityManagerService-কে অনুরোধ পাঠায়। এই service-ই broker।

Pub/Sub-এ	Android-এ
বার্তা	Intent
Topic	Intent-এর action string, যেমন `android.intent.action.BATTERY_CHANGED`
বার্তার বিষয়বস্তু	Intent-এর extras
Publisher	যে `sendBroadcast()` ডাকে, অথবা সিস্টেম নিজে
Subscriber	BroadcastReceiver, যার `onReceive()` বার্তা পায়
Subscribe / unsubscribe	`registerReceiver()` / `unregisterReceiver()`
Filter	IntentFilter
Broker	ActivityManagerService

একটি broadcast পাঁচটি ধাপে চলে:

১. **Subscribe:** `registerReceiver(receiver, filter)` সিস্টেমের তালিকায় লিখে রাখে, "এই receiver এই action-এর বার্তা চায়"।

২. **Publish:** `sendBroadcast(intent)` Intent-টি সিস্টেমের হাতে দিয়ে সঙ্গে সঙ্গে ফিরে আসে; প্রেরক অপেক্ষা করে না।

৩. **Route:** সিস্টেম Intent-টিকে নিবন্ধিত প্রতিটি IntentFilter-এর সঙ্গে মিলিয়ে উপযুক্ত receiver খুঁজে বের করে।

৪. **Deliver:** বার্তাটি receiver-এর প্রসেসে পৌঁছে সেই প্রসেসের main thread-এর message queue-তে জমা হয়। এখানেই pub/sub আর message queue মিলে যায়।

৫. **Consume:** main thread-এর Looper যখন বার্তাটিতে পৌঁছায়, তখন receiver-এর `onReceive()` চলে।

Receiver দুইভাবে subscribe করতে পারে। **Manifest-declared** receiver `AndroidManifest.xml`-এ লেখা একটি স্থায়ী subscription; দরকার হলে সিস্টেম অ্যাপ চালু করেও এতে বার্তা দিতে পারে, তবে Android 8 থেকে বেশিরভাগ implicit broadcast এভাবে আর পাঠানো হয় না। **Context-registered** receiver কোডে `registerReceiver()` দিয়ে তৈরি একটি অস্থায়ী subscription, যা শুধু register থেকে unregister পর্যন্ত বাঁচে এবং শেষে অবশ্যই unregister করতে হয়। এই অ্যাপের দুটি receiver-ই দ্বিতীয় ধরনের।

৩.৯ উদাহরণ ১: custom broadcast-এর পুরো যাত্রা

Activity 2-তে ব্যবহারকারী "Hello CSE489" লিখে Send চাপলেন। এরপর বার্তাটি কীভাবে receiver পর্যন্ত যায়, প্রতিটি ধাপ কোডের অবস্থানসহ:

১. **Point-to-point হস্তান্তর:** টেক্সটটি একটি explicit Intent-এর extra হয়ে Activity 3-এ যায়; এখানে এখনো কোনো broadcast নেই।

`BroadcastInputActivity.kt › sendMessage()` লাইন 82–90

২. **Subscriber তৈরি:** Activity 3 নিজের `CustomMessageReceiver` তৈরি করে।

`CustomReceiverActivity.kt › showCustomReceiver()` লাইন 74

৩. **Subscribe:** onStart-এ receiver register হয়; filter শুধু `ACTION_CUSTOM_MESSAGE` মেলায়।

`CustomReceiverActivity.kt › onStart()` লাইন 85–90

৪. **Publish:** subscribe করার পরেই Activity 3 Intent তৈরি করে (topic = `ACTION_CUSTOM_MESSAGE`, বিষয়বস্তু = টেক্সট, `setPackage()` দিয়ে শুধু এই অ্যাপ) এবং `sendBroadcast()` ডাকে।

`CustomReceiverActivity.kt › sendCustomBroadcast()` লাইন 114–117

`CustomMessageReceiver.kt › createBroadcast()` লাইন 46–49

৫. **Route:** ActivityManagerService নিবন্ধিত filter-গুলোর সঙ্গে action মিলিয়ে আমাদের receiver খুঁজে পায়। এটি Android নিজে করে, প্রজেক্টে এর কোড নেই।

৬. **Deliver:** বার্তাটি আমাদের প্রসেসের main thread-এর queue-তে জমা হয়।

৭. **Consume:** Looper পৌঁছালে `onReceive()` চলে: action যাচাই, টেক্সট বের করা, Log লেখা, তারপর callback।

`CustomMessageReceiver.kt › onReceive()` লাইন 23–28

৮. **ভেতরের observer-এ হস্তান্তর:** callback ViewModel-এ বার্তাটি রাখে, আর LiveData-র observer স্ক্রিনে "Received via broadcast: Hello CSE489" দেখায়।

`CustomReceiverActivity.kt` › **`onBroadcastReceived()`** লাইন 119–122

`CustomReceiverViewModel.kt` › **`onMessageReceived()`** লাইন 29–32

৯. **Unsubscribe:** স্ক্রিন থেকে চলে গেলে onStop-এ receiver unregister হয়।

`CustomReceiverActivity.kt` › **`onStop()`** লাইন 105–112

> **লক্ষ করার বিষয়**
> একটি বার্তা এখানে চার ধরনের পথ পার হলো: point-to-point (Intent দিয়ে Activity 3-এ), pub/sub (broadcast), message queue (main thread) এবং শেষে observer pattern (LiveData দিয়ে UI-তে)। আর এটি রেডিওর মতো non-durable: register-এর আগে পাঠালে বার্তাটি কেউ পেত না। সে কারণেই ধাপ ৩ আগে, ধাপ ৪ পরে।

৩.১০ উদাহরণ ২: ব্যাটারি, একটি retained বার্তা

ব্যাটারির ক্ষেত্রে publisher সিস্টেম নিজে, আর বার্তাটি নোটিশ বোর্ডের মতো retained:

১. সিস্টেমের battery service ব্যাটারির অবস্থা বদলালেই `ACTION_BATTERY_CHANGED` প্রকাশ করে, এবং শেষ বার্তাটি রেখে দেয় (sticky)। এটি Android-এর ভেতরে ঘটে।

২. Activity 2 দৃশ্যমান হলে `BatteryStatusLiveData`-র `onActive()` subscribe করে।

`BatteryStatusLiveData.kt` › **`onActive()`** লাইন 29–40

৩. `registerReceiver()` সঙ্গে সঙ্গে রেখে দেওয়া শেষ Intent-টি ফেরত দেয়, আর কোড সেখান থেকেই প্রথম মান বসায় লাইন 39 । তাই পরের পরিবর্তনের অপেক্ষা ছাড়াই স্ক্রিনে শতাংশ দেখা যায়, ঠিক যেমন দেরিতে আসা ছাত্র বোর্ডে সর্বশেষ নোটিশ দেখে।

৪. এরপর প্রতিটি নতুন broadcast-এ receiver-এর `onReceive()` মান হালনাগাদ করে।

`BatteryStatusLiveData.kt` › **`receiver`** লাইন 22–27

৫. স্ক্রিন অদৃশ্য হলে `onInactive()` unsubscribe করে।

`BatteryStatusLiveData.kt` › **`onInactive()`** লাইন 42–44

এই broadcast "protected": শুধু সিস্টেম এটি প্রকাশ করতে পারে, কোনো অ্যাপ জাল ব্যাটারি-বার্তা পাঠাতে পারে না। Pub/Sub-এর ভাষায়, topic-টির publisher-এর ওপর নিয়ন্ত্রণ আছে। বিস্তারিত **৭.৯** ও **৭.১০** অংশে।

৪ দুই ব্যবস্থার তুলনা ও এই অ্যাপের মানচিত্র

এই অধ্যায়ে message queue ও pub/sub পাশাপাশি রেখে তুলনা করব, তারপর একটি মানচিত্রে দেখব কোন ধারণা এই অ্যাপের কোন কোডে কাজ করছে।

৪.১ Message Queue বনাম Pub/Sub

বিষয়	Message Queue	Publish/Subscribe
একটি বার্তা কতজন পায়	ঠিক একজন consumer	topic-এর সব subscriber (fan-out)
প্রেরক কি প্রাপককে চেনে	queue-টি চেনে, consumer-কে নয়	শুধু topic-এর নাম জানে
প্রাপক তখন না থাকলে	বার্তা queue-তে অপেক্ষা করে	সাধারণত হারিয়ে যায়, যদি না retained বা durable হয়
প্রাপক বাড়ালে	কাজ ভাগ হয়, দ্রুত শেষ হয়	প্রত্যেকে আলাদা কপি পায়
মূল ব্যবহার	কাজ বণ্টন: "এই কাজটি কেউ একজন করো"	ঘটনা জানানো: "এটা ঘটেছে, যার দরকার জেনে নাও"
দৈনন্দিন উদাহরণ	রেস্তোরাঁর অর্ডার স্লিপ, ব্যাংকের টোকেন	YouTube notification, রেডিও, নোটিশ বোর্ড
Android-এ	main thread-এর Looper ও MessageQueue; explicit Intent	broadcast; ভেতরে LiveData ও StateFlow (observer)
এই অ্যাপে	touch, lifecycle, onPrepared, `delay(200)`	custom broadcast, ব্যাটারির broadcast

> **এক বাক্যে**
>
> **কাজ** পাঠাতে চাইলে queue, **খবর** ছড়াতে চাইলে pub/sub। Queue নিশ্চিত করে কাজটি একজনই একবার করবে; pub/sub খবরটি পৌঁছে দেয় সেই মুহূর্তে যারা শুনছে তাদের সবার কাছে, প্রেরককে তাদের না চিনেই।

৪.২ Explicit Intent: point-to-point বার্তা

Activity খোলার explicit Intent-ও এক ধরনের point-to-point বার্তা। প্রাপক নির্দিষ্ট (একটি নির্দিষ্ট Activity class), আর বার্তাটি হারায় না, কারণ সিস্টেম প্রাপক Activity তৈরি করে Intent-টি তার হাতেই দেয়; নতুন Activity যখনই তৈরি হোক, Intent সঙ্গে নিয়েই আসে। এই অ্যাপে প্রতিটি Activity-র একটি `newIntent()` factory আছে, যা বার্তার "খাম" তৈরি করে:

কোডে কোথায় `BroadcastInputActivity.kt` › **`newIntent()` (mode সহ)** লাইন 139–140

`CustomReceiverActivity.kt` › **`newIntent()` (mode ও টেক্সট সহ)** লাইন 167–170

ব্যবহারকারীর টেক্সট broadcast-এ নয়, explicit Intent-এ Activity 3-এ পাঠানোর কারণ এখানেই: point-to-point পথে বার্তা হারায় না, কিন্তু non-durable pub/sub-এ দেরিতে আসা subscriber বার্তা পায় না (**৭.৭**)।

৪.৩ বড় মানচিত্র: ধারণা থেকে কোড

নিচের তালিকা তত্ত্ব ও কোডের সেতু। বাম দিকে ধারণা, মাঝে Android-এ তার নাম, আর ডানে এই প্রজেক্টে ঠিক কোথায় সেটি আছে।

ধারণা	Android-এ	এই অ্যাপের কোডে
Topic	Intent-এর action	`CustomMessageReceiver.kt` › **`ACTION_CUSTOM_MESSAGE`** লাইন 34
Subscription filter	IntentFilter	`CustomMessageReceiver.kt` › **`intentFilter()`** লাইন 40
Subscribe	`registerReceiver()`	`CustomReceiverActivity.kt` › **`onStart()`** লাইন 85–90
Unsubscribe	`unregisterReceiver()`	`CustomReceiverActivity.kt` › **`onStop()`** লাইন 105–112
Publish	`sendBroadcast()`	`CustomReceiverActivity.kt` › **`sendCustomBroadcast()`** লাইন 114–117
Subscriber	BroadcastReceiver	`CustomMessageReceiver.kt` › **`onReceive()`** লাইন 23–28
প্রাপক সীমিত করা	`setPackage()`	`CustomMessageReceiver.kt` › **`createBroadcast()`** লাইন 46–49
কে পাঠাতে পারে	`RECEIVER_NOT_EXPORTED`; protected broadcast	`CustomReceiverActivity.kt` › **`onStart()`** লাইন 85–90
Retained বার্তা	Sticky broadcast	`BatteryStatusLiveData.kt` › **`onActive()`** লাইন 29–40
De-duplication	“আগে পাঠানো হয়েছে” flag	`CustomReceiverViewModel.kt` › **`initialBroadcastSent`** লাইন 22–26
Point-to-point বার্তা	Explicit Intent	`CustomReceiverActivity.kt` › **`newIntent()`** লাইন 167–170
Queue ও একজন consumer	Looper ও MessageQueue	সব callback; যেমন `ZoomableImageView.kt` › **`onTouchEvent()`** লাইন 170–187
Delayed বার্তা	`delay()`, `postDelayed()`	`AudioPlayerViewModel.kt` › **`startProgressUpdates()`** লাইন 125–133
Async request/reply	`prepareAsync()` → onPrepared	`AudioPlayerViewModel.kt` › **`init`** লাইন 64–69
ভেতরের pub/sub (observer)	LiveData, StateFlow	`CustomReceiverViewModel.kt` › **`lastReceived`** লাইন 19 `AudioPlayerViewModel.kt` › **`uiState`** লাইন 47–48

ধারণা	Android-এ	এই অ্যাপের কোডে
Lifecycle-বাঁধা subscription	onActive/onInactive; `repeatOnLifecycle`	`BatteryStatusLiveData.kt` › **`onActive() … onInactive()`** লাইন 29–44
Fan-out to listeners	NavController → toolbar ও drawer	`MainActivity.kt` › **`setUpNavigation()`** লাইন 59–71
একজন consumer-ই জেতে	OnBackPressedCallback	`MainActivity.kt` › **`closeDrawerOnBack`** লাইন 35–39

৪.৪ কোন ফিচারে কোন ব্যবস্থা

ফিচার	Message queue ও point-to-point	Pub/Sub ও observer
Navigation Drawer	Touch ও Back event; Back-এর dispatcher-এ একজন consumer	NavController-এর পরিবর্তন toolbar ও drawer-এ fan-out
Custom broadcast	Explicit Intent-এ টেক্সট; `onReceive()` main queue দিয়ে আসে	Custom broadcast (non-durable); প্রাপ্ত বার্তার LiveData
Battery broadcast	`onReceive()` main queue দিয়ে আসে	Sticky broadcast (retained); LiveData-র onActive/onInactive
Image Scale	Touch event-এর ধারা; animation-এর frame	ছবির StateFlow (Loading/Success/Error); `onZoomChanged` callback
Video	onPrepared, onError, onCompletion বার্তা	Drawer-এর listener (drawer নড়লে controls লুকানো)
Audio	`prepareAsync()` → onPrepared; `delay(200)`-এর লুপ	AudioUiState-এর StateFlow

৫ Lifecycle ও state

Subscribe করার সঠিক সময় কখন, আর স্ক্রিন ঘোরালে তথ্য কোথায় থাকে, এই দুটি প্রশ্নের উত্তর পুরো অ্যাপে একই নিয়মে দেওয়া হয়েছে। এই অধ্যায়ে সেই নিয়মগুলো।

৫.১ Lifecycle: কখন subscribe, কখন unsubscribe

Activity ও Fragment-কে সিস্টেম তৈরি করে, দেখায়, লুকায় ও ধ্বংস করে, এবং প্রতিটি ধাপ একটি lifecycle callback দিয়ে জানায়: onCreate → onStart → onResume → onPause → onStop → onDestroy। onStart থেকে onStop পর্যন্ত স্ক্রিন দৃশ্যমান (STARTED অবস্থা), আর onResume থেকে onPause পর্যন্ত সবার সামনে থেকে ইনপুট পায় (RESUMED অবস্থা)।

Callback-গুলো জোড়ায় জোড়ায় আসে। এই প্রজেক্টের মূলনীতি: **যে callback-এ subscribe, তার জোড়া callback-এ unsubscribe**, যাতে কোনো subscription তার স্ক্রিনের চেয়ে বেশি দিন না বাঁচে।

জোড়া	কী শুরু ও শেষ হয়	কোডে কোথায়
onStart ↔ onStop	Activity 3-এর custom receiver	`CustomReceiverActivity.kt` › **`onStart()`** লাইন 79–103 `CustomReceiverActivity.kt` › **`onStop()`** লাইন 105–112
onActive ↔ onInactive (LiveData)	ব্যাটারির receiver	`BatteryStatusLiveData.kt` › **`onActive()` … `onInactive()`** লাইন 29–44
`repeatOnLifecycle(STARTED)`	Image Scale ও Audio স্ক্রিনে StateFlow collect করা	`ImageScaleFragment.kt` › **`onViewCreated()`** লাইন 42–46 `AudioFragment.kt` › **`onViewCreated()`** লাইন 39–43
onViewCreated ↔ onDestroyView	Fragment-এর binding; Video স্ক্রিনের drawer listener	`VideoFragment.kt` › **`onViewCreated()`** লাইন 57–58 `VideoFragment.kt` › **`onDestroyView()`** লাইন 91–100
ViewModel init ↔ onCleared	Audio-র MediaPlayer তৈরি ও release	`AudioPlayerViewModel.kt` › **`init`** লাইন 53–70 `AudioPlayerViewModel.kt` › **`onCleared()`** লাইন 141–144

নিয়ম না মানলে কী হয়? ধরা যাক onCreate-এ receiver register করা হলো, কিন্তু unregister করা হলো না। সিস্টেম receiver-টিকে ধরে রাখে, আর receiver ধরে রাখে Activity-কে; ফলে বন্ধ হয়ে যাওয়া Activity মেমরিতে আটকে থাকে (memory leak), আর কেউ দেখছে না এমন স্ক্রিনের জন্যও `onReceive()` চলতেই থাকে। Messaging-এর ভাষায়: একটি পরিত্যক্ত subscription, যা তবুও বার্তা পেয়ে যাচ্ছে।

Fragment-এর ক্ষেত্রে আরেকটি সূক্ষ্মতা আছে: Fragment নিজে বেঁচে থাকলেও তার view ধ্বংস হতে পারে (যেমন back stack-এ থাকার সময়)। তাই UI-সংক্রান্ত observer-গুলো Fragment-এর নিজের lifecycle-এর সঙ্গে নয়, `viewLifecycleOwner`-এর সঙ্গে বাঁধা হয়।

৫.২ Configuration change, ViewModel ও saved state

স্ক্রিন ঘোরালে (rotation), dark mode বা ভাষা বদলালে Android সাধারণত চলমান Activity ধ্বংস করে নতুন করে তৈরি করে, যাতে নতুন configuration-এর উপযোগী resource লোড হয় (যেমন `layout-land`-এর landscape layout)। এতে Activity ও Fragment-এর ভেতরের সাধারণ ভেরিয়েবলগুলো হারিয়ে যায়। এই অ্যাপ `android:configChanges` দিয়ে ব্যাপারটি এড়িয়ে যায় না; বরং তিন স্তরে state রক্ষা করে:

১. **ViewModel** configuration change-এর মধ্যেও বেঁচে থাকে; স্ক্রিন স্থায়ীভাবে চলে গেলে তবেই `onCleared()` ডাকা হয়। তাই দীর্ঘস্থায়ী জিনিস, যেমন চলমান ডাউনলোড বা MediaPlayer, এখানে রাখা হয়। উদাহরণ: `AudioPlayerViewModel.kt › player` লাইন 50

২. **SavedStateHandle** হলো ViewModel-এর ভেতরের একটি key-value store, যার মান Bundle-এ সংরক্ষিত হয়। অ্যাপ ব্যাকগ্রাউন্ডে থাকার সময় সিস্টেম মেমরির জন্য প্রসেসটি মেরে ফেললেও (process death) ফিরে এলে মানগুলো আবার পাওয়া যায়। ছোট কিন্তু গুরুত্বপূর্ণ মান, যেমন Spinner-এর নির্বাচন, প্রাপ্ত বার্তা বা ভিডিওর অবস্থান, এখানে থাকে। উদাহরণ: `BroadcastViewModel.kt › selectedMode` লাইন 16–17

৩. **View-এর নিজস্ব saved state:** `onSaveInstanceState()` ও `onRestoreInstanceState()`। id থাকা view নিজের অবস্থা নিজেই রাখতে পারে, যেমন text field-এর লেখা বা zoom-এর মাত্রা। উদাহরণ: `ZoomableImageView.kt › onSaveInstanceState()` লাইন 268–275

ViewModel থেকে UI-তে মান যায় **observable** ধারকের মাধ্যমে: LiveData বা StateFlow। দুটিই আসলে "সর্বশেষ মানসহ একটি topic" (**৩.৭**): নতুন observer যুক্ত হওয়ামাত্র বর্তমান মানটি পায়, তারপর প্রতিটি পরিবর্তন পায়। Rotation-এর পর নতুন স্ক্রিন observe শুরু করামাত্র আগের মান ফিরে পায়, এটিই এর বড় সুবিধা।

শেষ হাতিয়ারটি হলো Activity-র `isChangingConfigurations`। onPause বা onStop-এর সময় এটি বলে দেয় স্ক্রিনটি কি সত্যিই চলে যাচ্ছে, নাকি শুধু ঘুরছে। ভিডিও ও অডিও স্ক্রিন এর ওপর নির্ভর করে ঠিক করে, থামবে নাকি চলতে থাকবে:

কোডে কোথায় `VideoFragment.kt › onPause()` লাইন 76–79 `AudioFragment.kt › onStop()` লাইন 52

৬ অ্যাপের কাঠামো: Navigation Drawer

সংশ্লিষ্ট ফাইল: `MainActivity.kt` · `res/layout/activity_main.xml` · `res/navigation/nav_graph.xml` · `res/menu/drawer_menu.xml` · `util/WindowInsetsExt.kt` · `AndroidManifest.xml`

চারটি ফিচারের আগে সেই কাঠামোটি দেখি, যা এগুলোকে ধরে রাখে: একটি Activity, একটি drawer এবং Navigation component।

৬.১ একটি Activity, চারটি Fragment

কোডে কোথায় `activity_main.xml` › **`DrawerLayout`** লাইন 7–49
`activity_main.xml` › **`NavHostFragment`** লাইন 32–39 `activity_main.xml` › **`NavigationView`** লাইন 42–48
`MainActivity.kt` › **`onCreate()`** লাইন 41–50

Drawer-এর চারটি স্ক্রিনের জন্য অ্যাপে একটিই host Activity আছে: `MainActivity`। চারটি স্ক্রিন (Broadcast Receiver, Image Scale, Video ও Audio) প্রতিটি একটি Fragment। `MainActivity`-র layout এভাবে সাজানো:

- সবার বাইরে **DrawerLayout**, যা বাম পাশ থেকে টেনে আনা drawer সামলায়।
- মূল কনটেন্টে ওপরে **MaterialToolbar** (app bar), তার নিচে **NavHostFragment** (একটি FragmentContainerView); বর্তমান Fragment এখানেই দেখানো হয়।
- পাশে **NavigationView**, যা drawer menu ও তার header (কোর্সের নাম) দেখায়।

এই single-activity কাঠামোর সুবিধা হলো toolbar ও drawer একবারই তৈরি হয়; স্ক্রিন বদলালে শুধু মাঝের Fragment বদলায়, আর Navigation component back stack, শিরোনাম ও হাইলাইট সামলায়। Broadcast flow-এর দ্বিতীয় ও তৃতীয় ধাপ অবশ্য আলাদা Activity (`BroadcastInputActivity` ও `CustomReceiverActivity`), কারণ assignment-এ এই ধাপগুলোর জন্য নির্দিষ্ট Activity চাওয়া হয়েছে।

৬.২ Navigation graph ও drawer menu-র সংযোগ

কোডে কোথায় `nav_graph.xml` › **`navigation graph`** লাইন 6–35
`drawer_menu.xml` › **`drawer menu`** লাইন 7–20 `MainActivity.kt` › **`setUpNavigation()`** লাইন 59–71

`nav_graph.xml`-এ চারটি fragment destination আছে। startDestination হলো `broadcastFragment` লাইন 10, তাই অ্যাপ খুললে প্রথমে Broadcast Receiver স্ক্রিন আসে। প্রতিটি destination-এর label-ই সেই স্ক্রিনে toolbar-এর শিরোনাম হয়।

`drawer_menu.xml`-এর প্রতিটি item-এর id হুবহু একটি destination-এর id-এর সমান। এই মিলটিই আসল কৌশল: NavigationUI এই id দেখেই বোঝে কোন item চাপলে কোন স্ক্রিনে যেতে হবে, এবং কোন স্ক্রিনে থাকলে কোন item হাইলাইট থাকবে। Menu-র group-এ `checkableBehavior="single"`, তাই একবারে একটিই item চিহ্নিত থাকে।

`setUpNavigation()` তিনটি কাজ করে:

১. NavHostFragment থেকে NavController নেয় (FragmentContainerView ব্যবহার করলে এভাবেই নিতে হয়)।

২. Drawer menu থেকে `AppBarConfiguration` তৈরি করে লাইন 68 । ফলে চারটি স্ক্রিনই **top-level destination**: কোনোটিতে Up arrow (←) নেই, সবগুলোতে hamburger আইকন (☰), যা drawer খোলে। Menu থেকে বানানোয় menu-তে নতুন item যোগ করলে এই তালিকাও নিজে থেকে মিলে যায়।

৩. `setupWithNavController()` দিয়ে toolbar-কে (শিরোনাম ও আইকন) এবং NavigationView-কে (item-এ ক্লিক ও হাইলাইট) NavController-এর সঙ্গে যুক্ত করে লাইন 69–70 ।

Messaging-এর দৃষ্টিতে NavController একটি publisher: destination বদলালে toolbar ও drawer, দুই listener-ই খবর পায় এবং নিজেদের হালনাগাদ করে (fan-out, **৩.১**)। এ কারণেই কোথাও আলাদা করে শিরোনাম বা হাইলাইট বদলানোর কোড লিখতে হয়নি।

এর একটি গুরুত্বপূর্ণ ফল আছে: এক drawer item থেকে আরেকটিতে যাওয়ার সময় NavigationUI আগের স্ক্রিনের state (তার ViewModel-সহ) সংরক্ষণ করে রাখে, যাতে ফিরে এলে স্ক্রিনটি আগের অবস্থায় পাওয়া যায়। এ কারণে Audio স্ক্রিন ছেড়ে গেলেও তার player বেঁচে থাকে, আর গান থামাতে আলাদা ব্যবস্থা লাগে (দ্রষ্টব্য **১০.৭**)।

৬.৩ Fragment ও ViewBinding

কোডে কোথায় `BroadcastFragment.kt › _binding` লাইন 20–21
`BroadcastFragment.kt › onDestroyView()` লাইন 69–72
`BroadcastFragment.kt › viewLifecycleOwner` লাইন 32

প্রতিটি স্ক্রিন তার layout-এর জন্য ViewBinding ব্যবহার করে। Build-এর সময় প্রতিটি layout থেকে একটি binding class তৈরি হয়, যেখানে id থাকা প্রতিটি view-এর জন্য একটি type-safe ফিল্ড থাকে। ফলে `findViewById()` লাগে না, আর ভুল id বা ভুল type-এর কারণে crash-এর ঝুঁকিও থাকে না।

Fragment-এ binding থাকে শুধু onViewCreated থেকে onDestroyView পর্যন্ত, আর onDestroyView-এ সেটি null করে দেওয়া হয়। কারণ Fragment তার view-এর চেয়ে বেশি দিন বাঁচতে পারে; binding ধরে রাখলে ধ্বংস হয়ে যাওয়া পুরোনো view-গুলো মেমরিতে আটকে থাকত। একইভাবে UI-এর observer-গুলো `viewLifecycleOwner` দিয়ে যুক্ত, যাতে view চলে গেলে observer-ও নিজে থেকে সরে যায়। চারটি Fragment-ই এই একই ধাঁচে লেখা; ওপরের চিপগুলো Broadcast স্ক্রিনের উদাহরণ।

৬.৪ Back চাপলে আগে drawer বন্ধ হয়

কোডে কোথায় `MainActivity.kt › closeDrawerOnBack` লাইন 35–39
`MainActivity.kt › setUpBackHandling()` লাইন 73–84
`MainActivity.kt › onRestoreInstanceState()` লাইন 52–57 `AndroidManifest.xml › manifest` লাইন 15

প্রত্যাশিত আচরণ হলো: drawer খোলা অবস্থায় Back চাপলে অ্যাপ থেকে বের না হয়ে শুধু drawer বন্ধ হবে। নতুন Android-এ (targetSdk 36 বা তার বেশি, Android 16+) পুরোনো `onBackPressed()` আর ডাকাই হয় না; Back সামলানোর স্বীকৃত উপায় হলো OnBackPressedDispatcher-এ একটি OnBackPressedCallback যোগ করা।

- `closeDrawerOnBack` নামের callback-টি শুরুতে নিষ্ক্রিয় থাকে।
- একটি DrawerListener drawer খুললে callback-টি সক্রিয় করে, বন্ধ হলে আবার নিষ্ক্রিয় করে।
- ফলে drawer খোলা থাকলে Back শুধু drawer বন্ধ করে; অন্য সব সময় callback নিষ্ক্রিয় থাকায় Back তার স্বাভাবিক কাজ করে, predictive back animation-সহ।
- Rotation-এর পর DrawerLayout খোলা drawer আবার খুলে দেয়, কিন্তু listener-কে জানায় না। তাই `onRestoreInstanceState()` -এ drawer খোলা কি না দেখে callback-এর অবস্থা মিলিয়ে নেওয়া হয়।

Messaging-এর ভাষায় Back একটি event, যা dispatcher সক্রিয় callback-গুলোর মধ্যে সবচেয়ে শেষে যোগ হওয়াটিকে দেয়: একজন consumer-ই জেতে, ঠিক message queue-এর মতো। Manifest-এ `enableOnBackInvokedCallback="true"` Android 13 থেকে 15-এও predictive back চালু করে, যাতে সব সংস্করণে আচরণ একই থাকে।

৬.৫ Edge-to-edge ও window insets

কোডে কোথায় `WindowInsetsExt.kt` › **`applyWindowInsetsPadding()`** লাইন 20–46
`MainActivity.kt` › **`applyWindowInsets()`** লাইন 86–95
`BroadcastInputActivity.kt` › **`includeIme`** লাইন 126–131
`AndroidManifest.xml` › **`adjustResize`** লাইন 41–43

targetSdk 35 বা তার বেশি হলে Android 15+ ডিভাইসে প্রতিটি অ্যাপ **edge-to-edge** আঁকা হয়: status bar ও navigation bar স্বচ্ছ, আর অ্যাপের window এদের নিচ পর্যন্ত বিস্তৃত। এই অ্যাপের প্রতিটি Activity শুরুতেই `enableEdgeToEdge()` ডাকে লাইন 43, যাতে পুরোনো Android-এও একই চেহারা হয়। এর মানে, প্রতিটি স্ক্রিনকে নিজের toolbar ও বাটনগুলো সিস্টেম বারের নিচ থেকে সরিয়ে আনতে হয়, নইলে সেগুলো ঢাকা পড়ে যায়।

Window insets হলো সিস্টেম বার, display cutout (ক্যামেরার খাঁজ) বা on-screen কীবোর্ড স্ক্রিনের কোন পাশে কতটা জায়গা ঢেকে রেখেছে তার মাপ। `applyWindowInsetsPadding()` নামের extension function বেছে নেওয়া দিকগুলোতে XML-এ দেওয়া মূল padding-এর সঙ্গে inset যোগ করে:

- মূল padding একবার মনে রাখে, যাতে inset বারবার এলেও padding জমে বাড়তে না থাকে।
- Inset consume করে না, তাই পাশের বা ভেতরের অন্য view-ও একই inset পায়।
- App bar ওপর ও দুই পাশের inset নেয়: তার রং status bar-এর নিচ পর্যন্ত যায়, কিন্তু লেখা থাকে বারের নিচে। Fragment-এর এলাকা নিচ ও দুই পাশের inset নেয়, কারণ portrait-এ navigation bar নিচে থাকে আর landscape-এ পাশে। Drawer-এর header নেয় ওপরের inset।
- দ্বিতীয় Activity-র text field-এর জন্য `includeIme = true`: কীবোর্ডের (IME) উচ্চতাও padding-এ যোগ হয়, তাই কীবোর্ড খুললেও text field ও Send বাটন তার ওপরে দেখা যায়। Manifest-এর `adjustResize` সিস্টেমকে কীবোর্ডের inset পাঠাতে বলে, কিন্তু edge-to-edge window নিজে আর ছোট হয় না; তাই inset-টি নিজেদেরই প্রয়োগ করতে হয়।

`ViewGroupCompat.installCompatInsetsDispatch()` পুরোনো ও নতুন Android-এ একইভাবে প্রতিটি child view-এর কাছে inset পৌঁছানো নিশ্চিত করে।

৭ ফিচার A: Broadcast Receiver

সংশ্লিষ্ট ফাইল (`broadcast` প্যাকেজ): `BroadcastFragment` · `BroadcastViewModel` · `BroadcastMode` · `BroadcastInputActivity` · `CustomReceiverActivity` · `CustomMessageReceiver` · `CustomReceiverViewModel` · `BatteryStatusLiveData` · `BatteryViewModel` · `BatteryStatus`

এটি অ্যাপের সবচেয়ে বড় ফিচার, আর অধ্যায় ৩-এর pub/sub-এর সরাসরি প্রয়োগ। তিনটি স্ক্রিন জুড়ে দুটি আলাদা পথ: একটিতে আমরা নিজেরাই broadcast পাঠাই ও গ্রহণ করি, অন্যটিতে সিস্টেমের পাঠানো ব্যাটারির broadcast শুনি।

৭.১ পুরো প্রবাহ এক নজরে

প্রথম স্ক্রিন (`BroadcastFragment`, drawer-এর "Broadcast Receiver") থেকে ব্যবহারকারী একটি ধরন বেছে Proceed চাপেন। তারপর পথ দুই ভাগ হয়।

Custom broadcast receiver বেছে নিলে

১. Activity 2 (`BroadcastInputActivity`) একটি text field দেখায়; ব্যবহারকারী একটি বার্তা লিখে "Send to receiver" চাপেন।

২. টেক্সটটি Intent-এর ভেতরে করে Activity 3-এ (`CustomReceiverActivity`) যায়।

৩. Activity 3 নিজের `CustomMessageReceiver` তৈরি ও register করে, তারপর টেক্সটটি broadcast করে।

৪. Receiver broadcast-টি গ্রহণ করে; স্ক্রিনে "Received via broadcast: …", সময় ও একটি Toast দেখা যায়।

System battery notification receiver বেছে নিলে

১. Activity 2 সিস্টেমের `ACTION_BATTERY_CHANGED` broadcast শোনে এবং ব্যাটারির শতাংশ, চার্জিং অবস্থা ও শেষ আপডেটের সময় লাইভ দেখায়।

২. "Next" চাপলে Activity 3 খোলে, যা assignment অনুযায়ী কিছুই করে না, শুধু সেটি জানিয়ে দেয়।

৭.২ প্রথম স্ক্রিন: Spinner ও Proceed

কোডে কোথায় `BroadcastFragment.kt` › **`setUpSpinner()`** লাইন 42–62
`BroadcastFragment.kt` › **`onViewCreated()`** লাইন 25–40
`BroadcastFragment.kt` › **`openSelectedActivity()`** লাইন 65–67
`BroadcastMode.kt` › **`BroadcastMode`** লাইন 10–24
`BroadcastViewModel.kt` › **`BroadcastViewModel`** লাইন 13–30 `arrays.xml` › **`broadcast_types`** লাইন 4–7

স্ক্রিনে আছে "Select a broadcast type" লেখা, একটি Spinner, একটি Proceed বাটন, এবং নির্বাচিত ধরনটি পরের স্ক্রিনে কী করবে তার একটি ছোট বর্ণনা।

- **Spinner-এর item:** `arrays.xml`-এর একটি string-array-তে ঠিক দুটি item আছে। `ArrayAdapter.createFromResource()` সেগুলো Spinner-এ বসায় লাইন 44–48, আর `item_spinner` layout বন্ধ ও খোলা (dropdown), দুই অবস্থার চেহারা ঠিক করে।

- `BroadcastMode` **enum:** দুটি মান, `CUSTOM` ও `BATTERY`, string-array-এর একই ক্রমে সাজানো। তাই Spinner-এর position থেকে সরাসরি mode পাওয়া যায় (`fromPosition()`), আর সীমার বাইরের position হলে আসে null।
- **নির্বাচন মনে রাখা:** ব্যবহারকারী যা বাছেন, তা `BroadcastViewModel`-এর SavedStateHandle-এ রাখা হয়। Spinner তৈরির সময় মনে রাখা নির্বাচনটি বসিয়ে দেওয়া হয় লাইন 54, তাই rotation-এর পর, এমনকি process death-এর পরও, আগের নির্বাচনই দেখা যায়।
- **বর্ণনা হালনাগাদ:** `selectedMode` একটি LiveData। Fragment এটি observe করে লাইন 32–39, আর mode বদলালেই বর্ণনার লেখা বদলে যায়।
- **Proceed:** `BroadcastInputActivity.newIntent()` একটি explicit Intent তৈরি করে, যার extra-তে mode-এর নাম (যেমন `"CUSTOM"`) থাকে, এবং `startActivity()` দিয়ে Activity 2 খোলে।

৭.৩ Explicit Intent: এক Activity থেকে আরেক Activity-তে

কোডে কোথায় `BroadcastInputActivity.kt › newIntent()` লাইন 139–140
`BroadcastInputActivity.kt › onCreate()` লাইন 42–50
`CustomReceiverActivity.kt › newIntent()` লাইন 167–170
`AndroidManifest.xml › exported="false"` লাইন 41–46

Explicit Intent-এ গন্তব্য Activity-র class সরাসরি বলে দেওয়া থাকে, তাই সিস্টেম ঠিক সেটিই খোলে। এটি point-to-point বার্তা (**৪.২**): একজন প্রেরক, একজন নির্দিষ্ট প্রাপক, আর বার্তা হারানোর প্রশ্ন নেই।

- প্রতিটি Activity-র companion object-এ একটি `newIntent()` factory আছে। Extra-র key-গুলো (`"mode"`, `"message"`) সেই Activity-র ভেতরে private, তাই বাইরে থেকে কেউ ভুল key লিখতে পারে না।
- Enum পাঠানো হয় তার নাম (String) হিসেবে, আর গ্রহণকারী `BroadcastMode.fromName()` দিয়ে ফিরিয়ে আনে। নাম অনুপস্থিত বা অজানা হলে null আসে; তখন Activity Log-এ সতর্কবার্তা লিখে `finish()` করে, ভুল ডেটায় crash হয় না।
- Activity 2 ও 3 manifest-এ `exported="false"`, তাই শুধু এই অ্যাপই এগুলো খুলতে পারে।
- Toolbar-এর Up আইকন শুধু `finish()` ডাকে, ফলে আগের স্ক্রিন তার আগের অবস্থাতেই ফিরে আসে।

৭.৪ Custom পথ, Activity 2: বার্তা নেওয়া

কোডে কোথায় `BroadcastInputActivity.kt › showCustomInput()` লাইন 55–79
`BroadcastInputActivity.kt › sendMessage()` লাইন 82–90

`BroadcastInputActivity`-র layout-এ দুটি অংশ আছে: custom input ও battery status। Intent-এ আসা mode অনুযায়ী একটি অংশ দৃশ্যমান করা হয় এবং toolbar-এর শিরোনাম বদলানো হয়। Custom অংশে আছে "Message to broadcast" text field ও "Send to receiver" বাটন।

- **পাঠানোর তিন উপায়, একই কাজ:** বাটন চাপা, on-screen কীবোর্ডের Send কী (`IME_ACTION_SEND`), অথবা hardware keyboard-এর Enter লাইন 63–77। Enter আসে `IME_NULL` হিসেবে, সঙ্গে একটি key

event, আর সেটি আসে দুবার: একবার key down, একবার key up। তাই শুধু key down-এ একবার পাঠানো হয়, কিন্তু দুটি event-ই consume করা হয়, যাতে বার্তা দুবার না যায়।

- **যাচাই:** `sendMessage()` লেখার দুই পাশের ফাঁকা অংশ ছেঁটে (trim) দেখে কিছু বাকি আছে কি না। ফাঁকা হলে text field-এর নিচে "Please type a message first" ত্রুটি দেখিয়ে থেমে যায়; আবার টাইপ শুরু করলেই ত্রুটিটি মুছে যায়।
- **বৈধ হলে:** এখানে কোনো broadcast পাঠানো হয় না। টেক্সটটি Intent-এর ভেতরে করে Activity 3-এ পাঠানো হয় লাইন 89। কেন এমন নকশা, তা **৭.৭**-এ ব্যাখ্যা করা হয়েছে।
- **কীবোর্ডের ওপরে রাখা:** Manifest-এর `adjustResize` সিস্টেমকে কীবোর্ডের inset পাঠাতে বলে, আর scroll view সেই inset নিজের padding-এ যোগ করে (**৬.৫**)। টাইপ করা লেখাটি rotation-এও টিকে থাকে, কারণ id থাকা text field নিজের লেখা নিজেই সংরক্ষণ করে।

৭.৫ CustomMessageReceiver: আমাদের subscriber

কোডে কোথায় `CustomMessageReceiver.kt › CustomMessageReceiver` লাইন 19–51
`CustomMessageReceiver.kt › onReceive()` লাইন 23–28
`CustomMessageReceiver.kt › ACTION_CUSTOM_MESSAGE, EXTRA_MESSAGE` লাইন 34–37
`CustomMessageReceiver.kt › intentFilter()` লাইন 40
`CustomMessageReceiver.kt › createBroadcast()` লাইন 46–49

`CustomMessageReceiver` হলো BroadcastReceiver-এর একটি subclass। এর কাজ একটাই: `onReceive()` এলে Intent থেকে টেক্সট বের করে সেটি constructor-এ পাওয়া callback-এ পৌঁছে দেওয়া। Receiver নিজে কোনো UI জানে না; টেক্সট পেয়ে কী করতে হবে তা ঠিক করে Activity। এতে দুটি অংশ আলাদা ও সহজে পরীক্ষাযোগ্য থাকে।

- **Topic-এর নাম:** `ACTION_CUSTOM_MESSAGE`-এর মান হলো অ্যাপের applicationId-এর পরে `.action.CUSTOM_MESSAGE`; টেক্সটের extra-র নাম `EXTRA_MESSAGE`-ও একইভাবে applicationId দিয়ে শুরু। এই prefix নামটিকে পুরো ডিভাইসে অনন্য রাখে, তাই অন্য কোনো অ্যাপের action-এর সঙ্গে ভুল করে মিলে যায় না।
- **Subscription filter:** `intentFilter()` এমন একটি IntentFilter দেয়, যা শুধু এই action মেলায়।
- **বার্তা তৈরি:** `createBroadcast()` action, টেক্সট extra এবং `setPackage(packageName)` দিয়ে broadcast Intent বানায়। `setPackage()` delivery এই অ্যাপের ভেতরেই সীমাবদ্ধ রাখে, তাই ব্যবহারকারীর লেখা অন্য কোনো অ্যাপ পায় না।
- **গ্রহণ:** `onReceive()` সতর্কতা হিসেবে আবার action মিলিয়ে দেখে, extra না থাকলে চুপচাপ ফিরে যায়, তারপর Logcat-এ "Received via broadcast: …" লিখে callback ডাকে।
- **Manifest-এ নেই কেন:** Assignment চায় receiver-টি তৃতীয় Activity-তে তৈরি হোক। তাছাড়া এটি শুধু ওই স্ক্রিন খোলা থাকলেই দরকার; manifest receiver স্থায়ী subscription, যার প্রয়োজন এখানে নেই।

`onReceive()` main thread-এ চলে (**২.৬**), তাই এটি দ্রুত শেষ হওয়া উচিত; দীর্ঘক্ষণ আটকে থাকলে ANR হতে পারে। এখানে শুধু একটি মান হালনাগাদ হয় আর একটি Toast দেখানো হয়, যা মুহূর্তের কাজ।

৭.৬ Activity 3: register, publish, receive

কোডে কোথায় `CustomReceiverActivity.kt › onCreate()` লাইন 37–62
`CustomReceiverActivity.kt › showCustomReceiver()` লাইন 64–77
`CustomReceiverActivity.kt › onStart()` লাইন 79–103
`CustomReceiverActivity.kt › onStop()` লাইন 105–112
`CustomReceiverActivity.kt › sendCustomBroadcast() … renderReceived()` লাইন 114–139

`CustomReceiverActivity`-তে পুরো pub/sub চক্রটি একটি স্ক্রিনেই ঘটে (একই যাত্রা pub/sub-এর দৃষ্টিতে ৩.৯-এ দেখানো হয়েছে)। ক্রম অনুযায়ী:

১. **onCreate:** Intent থেকে mode ও টেক্সট পড়ে। Custom mode-এ টেক্সট না থাকলে `finish()`। থাকলে স্ক্রিন সাজায় (receiver-এর action দেখায়, "Broadcast again" বাটন যুক্ত করে) এবং `CustomMessageReceiver` তৈরি করে, যার callback হলো `onBroadcastReceived()` লাইন 74।

২. **onStart, subscribe:** `ContextCompat.registerReceiver()` দিয়ে, `RECEIVER_NOT_EXPORTED` flag-সহ, receiver register হয় লাইন 85–90; স্ক্রিনে লেখা ওঠে "CustomMessageReceiver is registered"।

৩. **onStart, publish:** প্রথম broadcast আগে পাঠানো না হয়ে থাকলে "পাঠানো হয়েছে" flag সেট করে `sendBroadcast()` দিয়ে টেক্সটটি প্রকাশ করা হয় লাইন 99–102। `sendBroadcast()` সঙ্গে সঙ্গে ফিরে আসে; Activity কারও গ্রহণের জন্য অপেক্ষা করে না।

৪. **Route ও deliver:** সিস্টেম Intent-টিকে নিবন্ধিত IntentFilter-গুলোর সঙ্গে মিলিয়ে আমাদের receiver খুঁজে পায় এবং বার্তাটি আমাদের প্রসেসের main thread-এর queue-তে রাখে।

৫. **Consume:** Looper বার্তাটিতে পৌঁছালে `onReceive()` চলে, যা `onBroadcastReceived()` ডাকে লাইন 119–122। এটি ViewModel-কে টেক্সট ও সময় দেয় (ViewModel গ্রহণের সংখ্যা এক বাড়িয়ে সবকিছু SavedStateHandle-এ রাখে) এবং একটি Toast দেখায়।

৬. **UI হালনাগাদ:** প্রাপ্ত বার্তার LiveData-র observer স্ক্রিনে দেখায় "Received via broadcast: …", সময় এবং মোট কতবার পাওয়া গেছে লাইন 124–139। সংখ্যার সঙ্গে সঠিক বচনের জন্য plurals resource ব্যবহার করা হয়।

৭. **আবার পাঠানো:** "Broadcast again" একই টেক্সট আবার publish করে; প্রতিবার গণনা বাড়ে, যা প্রমাণ করে বার্তাটি প্রতিবার সত্যিই receiver-এর মধ্য দিয়ে আসছে।

৮. **onStop, unsubscribe:** `unregisterReceiver()` ডাকা হয় লাইন 108 এবং স্ক্রিনে "not registered" দেখায়।

দুটি বিষয় লক্ষ করার মতো। প্রথমত, onStart-এ ক্রমটি গুরুত্বপূর্ণ: আগে register, তারপর send। দ্বিতীয়ত, register ও unregister একটি lifecycle জোড়ায় (onStart ↔ onStop) থাকায় receiver কখনো leak হয় না, এবং স্ক্রিন অদৃশ্য থাকলে কোনো বার্তা প্রক্রিয়া হয় না।

> **RECEIVER_NOT_EXPORTED কেন**
> targetSdk 34 (Android 14) বা তার বেশি হলে, সিস্টেমের নয় এমন broadcast-এর জন্য কোডে receiver register করার সময় `RECEIVER_EXPORTED` বা `RECEIVER_NOT_EXPORTED` বলে দিতেই হয়; না দিলে SecurityException হয়। NOT_EXPORTED মানে অন্য কোনো অ্যাপ এই receiver-এ broadcast পাঠাতে পারবে না। `ContextCompat`-এর সংস্করণটি পুরোনো Android-এও একইভাবে কাজ করে।

৭.৭ হারিয়ে যাওয়া বার্তার সমস্যা

কোডে কোথায় `CustomReceiverActivity.kt` › **`onStart() (ব্যাখ্যাসহ মন্তব্য)`** লাইন 93–102

প্রথম দেখায় মনে হতে পারে, Activity 2-ই broadcast পাঠাবে আর Activity 3 সেটি গ্রহণ করবে। কিন্তু এই নকশা কাজ করে না।

কারণ: Activity 2 যখন `sendBroadcast()` ডাকত, তখনও Activity 3 তৈরিই হয়নি, তার receiver-ও register হয়নি। সাধারণ (non-sticky) broadcast সিস্টেম জমিয়ে রাখে না: পাঠানোর মুহূর্তে যারা subscribe করা আছে, শুধু তারাই পায়, আর পরে আসা subscriber কিছুই পায় না। এটি রেডিওর মতো non-durable delivery (**৩.৬**)। ফলে বার্তাটি নিঃশব্দে হারিয়ে যেত। তাছাড়া `startActivity()` ও `sendBroadcast()` দুটিই asynchronous, তাই "আগে Activity খুলি, তারপর পাঠাই" করলেও নিশ্চয়তা থাকত না যে receiver ততক্ষণে প্রস্তুত।

সমাধান দুই ধাপে:

১. টেক্সটটি Activity 2 থেকে Activity 3-এ যায় explicit Intent-এর ভেতরে, অর্থাৎ point-to-point পথে, যেখানে delivery নিশ্চিত।

২. Activity 3 আগে নিজের receiver register করে, তারপর একই টেক্সট broadcast করে। তাই broadcast পাঠানোর মুহূর্তে subscriber নিশ্চিতভাবেই উপস্থিত।

এভাবে assignment-এর শর্তও পূরণ হয়: টেক্সট নেওয়া হয় দ্বিতীয় Activity-তে, আর তৃতীয় Activity-তে তৈরি custom broadcast receiver সেটি গ্রহণ করে। Sticky broadcast (`sendStickyBroadcast()`) এখানে বিকল্প নয়, কারণ এটি deprecated এবং নিরাপত্তার কারণে নিরুৎসাহিত। Activity 3-এর স্ক্রিনে "Why does this screen send the broadcast?" অংশে এই ব্যাখ্যাটি ব্যবহারকারীর জন্যও লেখা আছে।

৭.৮ Rotation: বার্তা দুবার যায় না, কিছু হারায়ও না

কোডে কোথায় `CustomReceiverViewModel.kt` › **`CustomReceiverViewModel`** লাইন 16–38
`CustomReceiverViewModel.kt` › **`ReceivedMessage`** লাইন 45–49
`CustomReceiverActivity.kt` › **`onStart()`** লাইন 99–102

Rotation-এ Activity 3 ধ্বংস হয়ে নতুন করে তৈরি হয়, আর onStart আবার চলে। প্রতিবার onStart-এ broadcast পাঠালে প্রতিটি rotation-এ একটি বাড়তি বার্তা যেত এবং গণনা অকারণে বাড়ত। আবার Activity-র সাধারণ ভেরিয়েবলে রাখলে প্রাপ্ত বার্তাটিও মুছে যেত।

- **Duplicate ঠেকানো:** `initialBroadcastSent` একটি flag, যা ViewModel-এর SavedStateHandle-এ থাকে। প্রথমবার পাঠানোর আগেই এটি true করা হয়; পরে onStart flag দেখে আর পাঠায় না।

Messaging-এ একে de-duplication বলে: একই বার্তা একবারই প্রক্রিয়া করা (**২.৫**)।

- **কিছু না হারানো:** শেষ প্রাপ্ত বার্তা (টেক্সট, সময় ও গণনা নিয়ে একটি `ReceivedMessage`) একই SavedStateHandle-এ থাকে। নতুন Activity observe শুরু করামাত্র LiveData সেটি দিয়ে দেয়। `ReceivedMessage` Serializable, যাতে Bundle-এ রাখা যায় এবং process death-এর পরও ফিরে আসে।
- **Subscription:** Rotation-এর সময় পুরোনো Activity-র onStop-এ unregister এবং নতুনটির onStart-এ আবার register হয়; subscription স্ক্রিনের প্রতিটি জীবনের সঙ্গে বাঁধা।

৭.৯ Battery পথ: সিস্টেমের broadcast

কোডে কোথায় `BatteryStatusLiveData.kt` › **`onActive() (মন্তব্যসহ)`** লাইন 29–40
`BroadcastInputActivity.kt` › **`showBatteryStatus()`** লাইন 94–105

`Intent.ACTION_BATTERY_CHANGED` সিস্টেম পাঠায় যখনই ব্যাটারির অবস্থা বদলায়: চার্জের মাত্রা, চার্জার লাগানো বা খোলা, তাপমাত্রা ইত্যাদি। এখানে publisher সিস্টেমের battery service; আমাদের অ্যাপ শুধু subscriber। এই broadcast-এর তিনটি বিশেষ বৈশিষ্ট্য:

১. **Sticky:** সিস্টেম শেষ broadcast-টি রেখে দেয়, আর `registerReceiver()` নিজেই সেই সর্বশেষ Intent ফেরত দেয়। তাই পরের পরিবর্তনের অপেক্ষা না করেই স্ক্রিন সঙ্গে সঙ্গে মান দেখাতে পারে। Pub/Sub-এর ভাষায় এটি নোটিশ বোর্ডের মতো retained বার্তা (**৩.১০**)।

২. **Protected:** শুধু সিস্টেমই এটি পাঠাতে পারে; কোনো অ্যাপ জাল ব্যাটারি-বার্তা পাঠাতে পারে না। এ কারণে এটি Android 14-এর export flag নিয়মের বাইরে, তাই flag ছাড়াই register করা যায়।

৩. **শুধু কোডে register করা receiver পায়:** manifest-এ declare করা receiver এই broadcast কখনো পায় না।

Battery অংশে দেখানো হয় বড় করে শতাংশ, একটি গোল progress indicator, চার্জিং অবস্থা ("Status: Charging" ইত্যাদি), শেষ আপডেটের সময়, এবং একটি নোট যে স্ক্রিন দৃশ্যমান থাকলেই শুধু শোনা হয়। "Next" বাটন Activity 3 খোলে। Landscape-এর জন্য এই অংশের আলাদা layout আছে।

৭.১০ BatteryStatusLiveData: শুধু দৃশ্যমান থাকলে শোনা

কোডে কোথায় `BatteryStatusLiveData.kt` › **`BatteryStatusLiveData`** লাইন 17–45
`BatteryViewModel.kt` › **`BatteryViewModel`** লাইন 14–17
`BroadcastInputActivity.kt` › **`observe(this)`** লাইন 104

ব্যাটারির receiver সরাসরি Activity-তে না রেখে একটি বিশেষ LiveData-র ভেতরে রাখা হয়েছে: `BatteryStatusLiveData`। LiveData নিজেই জানে কখন তার অন্তত একজন **active** observer আছে (যার lifecycle অন্তত STARTED): প্রথম active observer এলে `onActive()`, আর শেষজন চলে গেলে `onInactive()` ডাকা হয়।

- **`onActive()`, subscribe:** application context দিয়ে `ACTION_BATTERY_CHANGED`-এর জন্য receiver register করা হয়; ফেরত পাওয়া sticky Intent থেকে সঙ্গে সঙ্গে প্রথম মান বসানো হয় লাইন 37–39।

- **`onReceive()`:** প্রতিটি নতুন broadcast-এ মান হালনাগাদ হয় লাইন 22–27। Context-registered receiver main thread-এ চলে, তাই সেখান থেকে সরাসরি LiveData-র মান বসানো নিরাপদ।
- **`onInactive()`, unsubscribe:** `unregisterReceiver()` লাইন 42–44।

Activity 2 নিজেকে lifecycle owner হিসেবে দিয়ে LiveData-টি observe করে। ফলে Activity-র onStart-এ LiveData active হয় (receiver register) আর onStop-এ inactive হয় (unregister)। Home চাপলে, Next দিয়ে Activity 3-এ গেলে বা rotation-এর সময় শোনা বন্ধ থাকে; ফিরে এলে আবার শুরু হয়, এবং sticky মান থেকে সঙ্গে সঙ্গে সর্বশেষ অবস্থা পাওয়া যায়। Activity নিজে register বা unregister নিয়ে কিছুই জানে না, শুধু observe করে।

- **Application context কেন:** এটি পুরো অ্যাপের সমান দীর্ঘজীবী, তাই LiveData কোনো Activity-কে ধরে রাখে না এবং leak হতে পারে না লাইন 20।
- **`BatteryViewModel`:** একটি AndroidViewModel, যা LiveData-র একটিমাত্র instance rotation-এর মধ্যেও ধরে রাখে; নতুন Activity observe শুরু করামাত্র শেষ মানটি পায়।
- **প্রয়োজনে তবেই তৈরি:** ViewModel-টি lazy-ভাবে তৈরি হয় লাইন 31, তাই custom পথে ব্যাটারির LiveData বা তার receiver কখনো তৈরিই হয় না।

৭.১১ Battery Intent থেকে শতাংশ ও অবস্থা

কোডে কোথায় `BatteryStatus.kt › percentOf()` লাইন 31–34
`BatteryStatus.kt › fromIntent()` লাইন 37–49 | `BatteryStatus.kt › ChargingStatus` লাইন 54–71
`BroadcastInputActivity.kt › renderBattery()` লাইন 107–120
`TimeFormat.kt › formatTimeOfDay()` লাইন 25–29

`ACTION_BATTERY_CHANGED` Intent-এর extras-এ অনেক তথ্য থাকে; এখানে তিনটি ব্যবহার করা হয়: `EXTRA_LEVEL` (বর্তমান মাত্রা), `EXTRA_SCALE` (সর্বোচ্চ মাত্রা) এবং `EXTRA_STATUS` (চার্জিং অবস্থা)। `BatteryStatus.fromIntent()` এগুলো পড়ে একটি `BatteryStatus` data class তৈরি করে।

- **শতাংশের সূত্র:** শতাংশ = `EXTRA_LEVEL` × ১০০ ÷ `EXTRA_SCALE`। Level-কে সরাসরি শতাংশ ধরে নেওয়া ভুল, কারণ সব ডিভাইসে scale ১০০ নয়।
- **সুরক্ষা:** `percentOf()` level অনুপস্থিত হলে (তখন `getIntExtra()` default হিসেবে −১ দেয়) বা scale শূন্য বা ঋণাত্মক হলে null দেয়, যাতে শূন্য দিয়ে ভাগ না হয়; স্ক্রিনে তখন "—" দেখায়। ফলাফল সবসময় ০ থেকে ১০০-এর মধ্যে আটকে রাখা হয়।
- **চার্জিং অবস্থা:** `EXTRA_STATUS`-এর মান (charging, discharging, full, not charging) `ChargingStatus` enum-এ রূপান্তরিত হয়; অপ্রত্যাশিত যেকোনো মান হয় `UNKNOWN`। প্রতিটি enum মানের সঙ্গে স্ক্রিনে দেখানোর লেখার string resource যুক্ত।
- **সময়:** `receivedAtMillis` হলো অ্যাপ কখন broadcast-টি পড়ল। `formatTimeOfDay()` ব্যবহারকারীর ১২ বা ২৪ ঘণ্টার সেটিং মেনে সেকেন্ডসহ সময় দেখায়, যাতে প্রতিটি লাইভ আপডেট চোখে পড়ে।
- **Progress indicator:** শতাংশ অনুযায়ী animation-সহ গোল ring পূরণ হয়; মান অজানা হলে ০।

শতাংশ ও অবস্থা নির্ণয়ের হিসাব UI থেকে আলাদা ছোট function-এ রাখা, তাই unit test দিয়ে প্রান্তিক ক্ষেত্রগুলো (scale শূন্য, level অনুপস্থিত) যাচাই করা হয়েছে। Emulator-এর battery control বা adb দিয়ে মাত্রা বদলালে স্ক্রিনের মান সঙ্গে সঙ্গে বদলায়।

৭.১২ Battery পথ, Activity 3: কিছুই না করা

কোডে কোথায় `CustomReceiverActivity.kt` › **`showNothingToDo()`** লাইন 147–150
`CustomReceiverActivity.kt` › **`onStart()`** লাইন 81

Assignment বলে, battery বেছে নিলে তৃতীয় Activity-তে কিছু করার নেই। `CustomReceiverActivity` এই mode-এ কোনো receiver তৈরি বা register করে না; receiver ভেরিয়েবল null থাকে, তাই onStart ও onStop কিছুই করে না। স্ক্রিনে শুধু "Nothing to do here" এবং কারণটি লেখা থাকে: লাইভ শতাংশ আগের স্ক্রিনেই দেখানো হয়।

৭.১৩ নিরাপত্তা: কে পাঠাতে পারে, কে শুনতে পারে

Broadcast একটি খোলা ব্যবস্থা, তাই pub/sub-এর প্রতিটি দিক আলাদাভাবে সুরক্ষিত করা হয়েছে:

ব্যবস্থা	কী নিয়ন্ত্রণ করে	কোডে কোথায়
`setPackage()`	Custom broadcast কে পাবে: শুধু এই অ্যাপের receiver।	`CustomMessageReceiver.kt` › **`createBroadcast()`** লাইন 46–49
`RECEIVER_NOT_EXPORTED`	কে এই receiver-এ পাঠাতে পারে: শুধু এই অ্যাপ (এবং সিস্টেম)।	`CustomReceiverActivity.kt` › **`onStart()`** লাইন 85–90
Protected broadcast	`ACTION_BATTERY_CHANGED` কে পাঠাতে পারে: শুধু সিস্টেম।	`BatteryStatusLiveData.kt` › **`onActive()`** লাইন 30–38
applicationId prefix	Custom action-এর নাম অনন্য রাখে; অন্য অ্যাপের নামের সঙ্গে সংঘাত হয় না।	`CustomMessageReceiver.kt` › **`ACTION_CUSTOM_MESSAGE`** লাইন 34
`exported="false"`	Activity 2 ও 3 অন্য অ্যাপ সরাসরি খুলতে পারে না; শুধু launcher-এর `MainActivity` exported।	`AndroidManifest.xml` › **`manifest`** লাইন 25–46

৮ ফিচার B: Image Scale

সংশ্লিষ্ট ফাইল (`image` প্যাকেজ): `ImageScaleFragment` · `ImageScaleViewModel` · `ZoomableImageView` · `ZoomMath` · `res/layout/fragment_image_scale.xml`

এই স্ক্রিনে দুটি আলাদা কাজ: ইন্টারনেট থেকে একটি ছবি নামানো, এবং pinch gesture দিয়ে সেটি zoom করা। Zoom-এর জন্য কোনো লাইব্রেরি নেই; touch event থেকে শুরু করে matrix-এর গণিত পর্যন্ত সবকিছু নিজস্ব `ZoomableImageView`-এ লেখা।

৮.১ ইন্টারনেট থেকে ছবি: Coil 3

কোডে কোথায় `ImageScaleViewModel.kt › load()` লাইন 45–64
`ImageScaleViewModel.kt › ImageUiState` লাইন 20–24 | `ImageScaleViewModel.kt › init` লাইন 40–42
`ImageScaleViewModel.kt › IMAGE_URL` লাইন 68 | `AndroidManifest.xml › INTERNET permission` লাইন 6

ছবিটি Lorem Picsum থেকে নেওয়া ২০০০ × ১৩০০ পিক্সেলের একটি ছবি। ছবি নামাতে ব্যবহার হয় Coil 3, একটি Kotlin coroutine-ভিত্তিক image loading লাইব্রেরি। নেটওয়ার্ক থেকে নামানো (OkHttp দিয়ে), decode করা এবং memory ও disk cache, সব এটি নিজে সামলায় এবং ভারী কাজগুলো background thread-এ করে।

- **Request:** `load()` একটি ImageRequest তৈরি করে, সঙ্গে `Size.ORIGINAL` লাইন 54। Coil সাধারণত ছবিকে view-এর আকারে ছোট করে decode করে, কিন্তু সেই ছোট bitmap-এ zoom করলে ছবি ঝাপসা দেখাত। তাই মূল রেজোলিউশন চাওয়া হয়, যাতে zoom করলেও ছবি যথেষ্ট পরিষ্কার থাকে।
- **Asynchronous request/reply:** ViewModel-এর scope-এ চলা একটি coroutine-এ `imageLoader.execute(request)` ডাকা হয় লাইন 56। এটি একটি suspend function: request পাঠিয়ে coroutine থেমে অপেক্ষা করে, কিন্তু main thread আটকায় না; উত্তর এলে coroutine যেখানে থেমেছিল সেখান থেকেই আবার চলে। Message queue-এর ভাষায়: ধীর কাজ অন্য thread-এ, ফল আবার main thread-এ (**২.৮**)।
- **ফলাফল একটি state-এ:** `ImageUiState` একটি sealed interface, যার তিনটি রূপ: Loading, Success (ছবিসহ) ও Error। Coil সফল হলে Success, ব্যর্থ হলে Log-এ সতর্কবার্তা লিখে Error। এই state থাকে একটি StateFlow-এ, যা UI-এর জন্য একটি retained topic (**৩.৭**)।
- **Rotation-এ আবার ডাউনলোড নয়:** `load()` চলে ViewModel তৈরির সময়। Rotation-এ ViewModel বেঁচে থাকে, তাই ডাউনলোড আবার শুরু হয় না; নতুন Fragment শুধু বিদ্যমান state collect করে।
- **Retry:** `load()` আগের job বাতিল করে, state-কে আবার Loading করে এবং নতুন করে চেষ্টা করে।

৮.২ স্ক্রিনে state দেখানো

কোডে কোথায় `ImageScaleFragment.kt › onViewCreated()` লাইন 32–47
`ImageScaleFragment.kt › render()` লাইন 49–60 | `ImageScaleFragment.kt › shownImage` লাইন 30

`ImageScaleFragment` state collect করে `repeatOnLifecycle(STARTED)`-এর ভেতরে: onStart-এ collect শুরু, onStop-এ বাতিল; অর্থাৎ স্ক্রিন দেখা গেলেই শুধু subscribe। প্রতিটি নতুন state-এ `render()` চলে:

- Loading হলে একটি progress indicator;
- Error হলে "Couldn't load the image…" বার্তা ও Retry বাটন;
- Success হলে ছবি, বর্তমান zoom-এর লেবেল (যেমন "1.0x") এবং hint "Pinch to zoom · double-tap to reset"।

একটি সূক্ষ্ম বিষয় আছে: StateFlow প্রতিটি নতুন collector-কে সর্বশেষ মানটি আবার দেয়, যেমন অন্য অ্যাপ থেকে ফিরে এলে onStart-এ। একই ছবি আবার view-এ বসালে zoom রিসেট হয়ে যেত। তাই Fragment বর্তমান ছবিটি (`shownImage`) মনে রাখে এবং শুধু সত্যিই নতুন ছবি এলে বসায়। Zoom বদলালে `ZoomableImageView`-এর `onZoomChanged` callback লেবেলটি হালনাগাদ করে।

৮.৩ Image Matrix ও "fit"

কোডে কোথায় `ZoomableImageView.kt › init (scaleType = MATRIX)` লাইন 70–73
`ZoomableImageView.kt › refit()` লাইন 104–118 `ZoomMath.kt › fitScale()` লাইন 22–23
`ZoomableImageView.kt › setImageDrawable()` লাইন 80–83

`ZoomableImageView` হলো AppCompatImageView-এর subclass, যার scaleType হলো `MATRIX`। এই মোডে ImageView ছবিকে কোথায়, কত বড় করে আঁকবে তা পুরোপুরি একটি **Matrix** ঠিক করে।

Matrix একটি ৩×৩ গাণিতিক রূপান্তর (affine transformation), যা ছবির (drawable-এর) প্রতিটি বিন্দুর স্থানাঙ্ককে view-এর স্থানাঙ্কে নিয়ে যায়। এতে scale (বড়-ছোট করা) ও translate (সরানো), দুটিই থাকে। `postScale()` ও `postTranslate()` বর্তমান রূপান্তরের পরে নতুন একটি রূপান্তর জুড়ে দেয়। `mapRect()` ও `mapPoints()` দিয়ে জানা যায় ছবির কোনো বিন্দু বা আয়তক্ষেত্র view-এর কোথায় আঁকা হচ্ছে, আর inverse matrix দিয়ে উল্টো দিকে, অর্থাৎ view-এর বিন্দু থেকে ছবির বিন্দুতে, যাওয়া যায়।

- **Fit:** পুরো ছবি বিকৃতি ছাড়া view-এর ভেতরে দেখাতে scale হয় (view-এর প্রস্থ ÷ ছবির প্রস্থ) এবং (view-এর উচ্চতা ÷ ছবির উচ্চতা), এই দুইয়ের মধ্যে যেটি ছোট, অর্থাৎ যে দিকে জায়গা কম সেদিক অনুযায়ী। তারপর ছবিকে মাঝখানে আনতে translate। এটি ImageView-এর FIT_CENTER-এর মতোই।
- **Zoom-এর সংজ্ঞা:** zoom = বর্তমান scale ÷ fit scale। তাই zoom ১ মানে পুরো ছবি দেখা যাচ্ছে; ২.৫ মানে fit-এর আড়াই গুণ বড়।
- **আবার fit (`refit()`):** ছবি বা view-এর আকার বদলালেই fit নতুন করে হিসাব হয়। এটি জরুরি, কারণ ছবি নেটওয়ার্ক থেকে asynchronous-ভাবে আসে, সাধারণত view-এর প্রথম layout-এর পরে। ছবি ও আকার দুটোই না জানা পর্যন্ত `refit()` কিছুই করে না।
- **Padding:** সব হিসাব view-এর padding বাদ দিয়ে আঁকার এলাকার ওপর করা হয়।

৮.৪ Pinch zoom: ScaleGestureDetector

কোডে কোথায় `ZoomableImageView.kt › ScaleListener` লাইন 196–211
`ZoomableImageView.kt › zoomBy()` লাইন 126–133 `ZoomMath.kt › clampZoom()` লাইন 16
`ZoomableImageView.kt › scaleDetector` লাইন 58–61

ScaleGestureDetector হলো Android-এর তৈরি একটি gesture detector, যা দুই আঙুলের touch event বিশ্লেষণ করে প্রতি মুহূর্তে দুটি জিনিস দেয়: **scaleFactor**, অর্থাৎ আগের event থেকে দুই আঙুলের দূরত্ব কত গুণ বদলেছে (১-এর বেশি হলে আঙুল ছড়াচ্ছে), এবং **focal point**, অর্থাৎ দুই আঙুলের মধ্যবিন্দু।

১. `onScaleBegin()`: শুরুর focal point মনে রাখে।

২. `onScale()`: প্রথমে focal point যতটা সরেছে ছবিকেও ততটা সরায়, ফলে দুই আঙুলে টেনেও ছবি সরানো যায়। তারপর `zoomBy()` ডাকে, scaleFactor ও focal point দিয়ে।

৩. `zoomBy()`: লক্ষ্য zoom = বর্তমান zoom × factor, যা ১x থেকে ৫x-এর মধ্যে আটকানো। আসলে প্রয়োগ করা হয় লক্ষ্য ÷ বর্তমান zoom; সীমায় পৌঁছালে এই অনুপাত ১ হয়ে যায়, তাই ছবি আর বড় বা ছোট হয় না। এরপর focal point ঘিরে `postScale()`, তারপর কিনারা ঠিক করা ও matrix প্রয়োগ।

Focal point ঘিরে scale কেন: কোনো বিন্দু f-কে কেন্দ্র করে s গুণ scale করলে ছবির যেকোনো বিন্দু p চলে যায় f + s × (p − f)-এ। p যদি ঠিক f হয়, তবে সে f-এই থাকে। অর্থাৎ আঙুলের নিচে ছবির যে অংশ ছিল, zoom-এর পরেও সেটি আঙুলের নিচেই থাকে; ব্যবহারকারীর মনে হয় তিনি ছবির ঠিক সেই জায়গাটিই টেনে বড় করছেন।

Detector-এর quick scale (double-tap করে আঙুল টেনে zoom) বন্ধ রাখা হয়েছে, কারণ এটি double-tap gesture-এর সঙ্গে সংঘাত করত। Zoom শুধু দুই আঙুলের pinch দিয়েই হয়।

৮.৫ এক আঙুলে pan ও কিনারা সামলানো

কোডে কোথায় `ZoomableImageView.kt › onScroll()` লাইন 216–221
`ZoomableImageView.kt › panBy()` লাইন 135–139
`ZoomableImageView.kt › clampTranslation()` লাইন 142–150
`ZoomMath.kt › edgeCorrection()` লাইন 31–39

দ্বিতীয় detector, GestureDetector, এক আঙুলের scroll, double-tap ও single tap চেনে। `onScroll()` জানায় আগের অবস্থান থেকে আঙুল কতটা সরেছে (উল্টো চিহ্নে), তাই চিহ্ন উল্টে নিয়ে ছবিকে ঠিক ততটা সরানো হয়; ফলে ছবি আঙুলের সঙ্গে সঙ্গে চলে। Pan হয় শুধু তখনই, যখন zoom ১-এর বেশি এবং pinch চলছে না; ১x-এ পুরো ছবি আগেই দেখা যাচ্ছে, সরানোর কিছু নেই।

প্রতিটি zoom বা pan-এর পর `clampTranslation()` চলে। এটি ছবির চার কোনা matrix দিয়ে view-এর স্থানাঙ্কে রূপান্তর করে, তারপর আনুভূমিক ও উল্লম্ব, প্রতিটি অক্ষে আলাদাভাবে `edgeCorrection()` হিসাব করে:

- ছবি ওই অক্ষে view-এর চেয়ে ছোট হলে: মাঝখানে আনা;
- বড় হলে এবং শুরুর দিকে (বাম বা ওপরে) ফাঁক থাকলে: ফাঁক বন্ধ হওয়া পর্যন্ত টেনে আনা;
- শেষের দিকে (ডান বা নিচে) ফাঁক থাকলে: সেদিকে টেনে আনা;
- নইলে কিছু না করা।

ফলে zoom করা ছবি যতই সরানো হোক, কিনারায় কখনো খালি ফাঁক দেখা যায় না। `ZoomMath`-এর এই function-গুলো Android-এর কোনো type ব্যবহার করে না, তাই সাধারণ JVM unit test দিয়ে যাচাই করা যায়; view-এর কোড শুধু event ও matrix সামলায়।

৮.৬ Double-tap

কোডে কোথায় `ZoomableImageView.kt` › **`onDoubleTap()`** লাইন 223–227
`ZoomableImageView.kt` › **`animateZoomTo()`** লাইন 158–168
`ZoomableImageView.kt` › **`DOUBLE_TAP_ZOOM, ZOOM_EPSILON, ZOOM_ANIMATION_MS`** লাইন 294–296

- **লক্ষ্য ঠিক করা:** `onDoubleTap()`-এ বর্তমান zoom ১x-এর চেয়ে বেশি হলে লক্ষ্য ১x (রিসেট), নইলে ২.৫x, tap করা বিন্দুকে কেন্দ্র করে। "বেশি" যাচাইয়ে ০.০১-এর একটি ছোট সহনশীলতা রাখা হয়, কারণ float হিসাবে zoom ঠিক ১.০ না হয়ে সামান্য বেশি হতে পারে।
- **মসৃণ animation:** একটি ValueAnimator ২৫০ মিলিসেকেন্ড ধরে বর্তমান zoom থেকে লক্ষ্য পর্যন্ত মান তৈরি করে, DecelerateInterpolator দিয়ে (শুরুতে দ্রুত, শেষে ধীর)। প্রতিটি frame-এ একই `zoomBy()` ডাকা হয়, তাই সীমা ও কিনারার নিয়ম animation-এর সময়ও বহাল থাকে। প্রতিটি frame main thread-এর queue-তে একটি করে বার্তা।
- **নিয়ন্ত্রণ ব্যবহারকারীর হাতে:** নতুন touch শুরু হলে চলমান animation সঙ্গে সঙ্গে বাতিল হয় লাইন 172। View window থেকে সরে গেলেও animation বাতিল হয়।

৮.৭ Touch event: একটি বার্তা-প্রবাহ

কোডে কোথায় `ZoomableImageView.kt` › **`onTouchEvent()`** লাইন 170–187
`ZoomableImageView.kt` › **`requestDisallowInterceptTouchEvent`** লাইন 178–180

আঙুলের প্রতিটি নড়াচড়া একটি MotionEvent হিসেবে main thread-এর queue দিয়ে আসে (**২.৭**): `ACTION_DOWN` (প্রথম আঙুল), `ACTION_POINTER_DOWN` (দ্বিতীয় আঙুল), অনেকগুলো `ACTION_MOVE`, তারপর `ACTION_POINTER_UP` ও `ACTION_UP`। একটি gesture আসলে এই বার্তাগুলোর একটি ধারা।

- `onTouchEvent()` প্রতিটি event দুই detector-কেই দেয়; প্রত্যেকে ধারাটি থেকে নিজের gesture (pinch, scroll বা double-tap) চিনে নেয়।
- true ফেরত দেওয়া মানে event-টি consume করা: এই view-ই gesture-এর মালিক। Message queue-এর মতো, একটি event একজনই consume করে।
- **Parent-কে আটকানো:** DrawerLayout পাশ থেকে টানাকে drawer খোলার জন্য নিজের দিকে নিতে চায়। Zoom ১-এর বেশি হলে বা একাধিক আঙুল থাকলে `requestDisallowInterceptTouchEvent(true)` ডাকা হয়, ফলে parent এই gesture কেড়ে নিতে পারে না এবং pan ও pinch নির্বিঘ্নে চলে।
- **Accessibility:** সাধারণ একক tap-এ এখনো `performClick()` ডাকা হয়, যা TalkBack-এর মতো accessibility service-এর জন্য দরকারি।
- ছবি না থাকলে view সাধারণ ImageView-এর মতো আচরণ করে।

৮.৮ Rotation-এ zoom ধরে রাখা

কোডে কোথায় `ZoomableImageView.kt › captureState()` লাইন 243–256
`ZoomableImageView.kt › restore()` লাইন 259–266
`ZoomableImageView.kt › onSaveInstanceState()` লাইন 268–275
`ZoomableImageView.kt › onRestoreInstanceState()` লাইন 277–291
`ZoomableImageView.kt › onSizeChanged()` লাইন 85–98

Rotation-এ Fragment-এর view নতুন করে তৈরি হয়, অর্থাৎ নতুন একটি `ZoomableImageView` আসে, আর তার আকারও বদলায় (portrait থেকে landscape)। শুধু zoom-এর সংখ্যা রাখলে যথেষ্ট নয়; ছবির কোন অংশটি দেখা যাচ্ছিল, তাও রাখতে হয়।

- **কী রাখা হয়:** zoom, এবং view-এর ঠিক মাঝখানে ছবির কোন বিন্দু ছিল, ছবির প্রস্থ ও উচ্চতার ভগ্নাংশ (০ থেকে ১) হিসেবে। পিক্সেলে না রেখে ভগ্নাংশে রাখায় ভিন্ন আকারের view-তেও ছবির একই অংশ আবার দেখানো যায়।
- **মাপা:** `captureState()` inverse matrix দিয়ে view-এর কেন্দ্রবিন্দুকে ছবির স্থানাঙ্কে নিয়ে যায়, তারপর ছবির মাপ দিয়ে ভাগ করে।
- **সংরক্ষণ:** `onSaveInstanceState()` parent class-এর state-এর সঙ্গে zoom ও কেন্দ্রের দুই ভগ্নাংশ একটি Bundle-এ রাখে। View-এর id থাকায় সিস্টেম নিজেই এটি ডাকে।
- **ফিরিয়ে আনা:** `onRestoreInstanceState()` মানগুলো সঙ্গে সঙ্গে প্রয়োগ না করে অপেক্ষমাণ (pending) হিসেবে রাখে, কারণ তখনও ছবি বা view-এর আকার না-ও জানা থাকতে পারে। `refit()` যখন দুটোই পায়, তখন আগে fit করে, তারপর সংরক্ষিত zoom প্রয়োগ করে, শেষে সংরক্ষিত বিন্দুটি view-এর মাঝখানে আনতে pan করে (`restore()`)।
- **আকার বদল:** আগে থেকেই আঁকা view-এর আকার বদলালে (যেমন multi-window-এ) `onSizeChanged()` পুরোনো আকারের হিসাবে state নিয়ে নতুন আকারে আবার প্রয়োগ করে; zoom ও কেন্দ্র অটুট থাকে।

এদিকে ছবিটি নিজে ViewModel-এর StateFlow-এ থাকায় rotation-এর পর আবার ডাউনলোড হয় না; নতুন view সঙ্গে সঙ্গে ছবি পায়, আর `refit()` সংরক্ষিত zoom ফিরিয়ে আনে।

৯ ফিচার C: Video

সংশ্লিষ্ট ফাইল (`video` প্যাকেজ): `VideoFragment` · `VideoViewModel` · `res/layout/fragment_video.xml` · `res/raw/sample_video.mp4`

ভিডিও চালানো সহজ শোনালেও আসল চ্যালেঞ্জ হলো স্ক্রিনের lifecycle-এর সঙ্গে player-কে মেলানো: rotation-এ চলতে থাকা, স্ক্রিন ছাড়লে থামা, আর ফিরে এলে ঠিক একই জায়গা থেকে শুরু করা।

৯.১ VideoView ও MediaController

কোডে কোথায় `VideoFragment.kt › onViewCreated()` লাইন 40–59
`VideoFragment.kt › setMediaController()` লাইন 50 `VideoFragment.kt › setVideoURI()` লাইন 54

VideoView হলো একটি SurfaceView, যার ভেতরে `android.media.MediaPlayer` কাজ করে: ভিডিও decode করে প্রতিটি frame surface-এ আঁকে এবং শব্দ বাজায়। ভিডিওর aspect ratio অনুযায়ী নিজের আকার ঠিক করে, তাই ছবি টেনে লম্বা বা চ্যাপ্টা হয় না।

ভিডিও ফাইলটি অ্যাপের ভেতরেই থাকে: `res/raw/sample_video.mp4` (Blender Foundation-এর Sintel ট্রেলার)। তাই ইন্টারনেট লাগে না। VideoView-কে এর ঠিকানা দেওয়া হয় একটি `android.resource://` URI হিসেবে, যেখানে অ্যাপের package নাম ও resource-এর id থাকে।

MediaController হলো Android-এর তৈরি নিয়ন্ত্রণ প্যানেল: play/pause, আগে-পিছে যাওয়া, seek bar, এবং চলমান ও মোট সময়। `setMediaController()` দিয়ে যুক্ত করলে VideoView এটিকে নিজের parent-এর (কালো ফ্রেমের) নিচের কিনারায় বসায়। MediaController নিজে কোনো ভিডিও জানে না; সে VideoView-কে MediaPlayerControl interface-এর মাধ্যমে চালায়: start, pause, seek, বর্তমান অবস্থান জানা ইত্যাদি। ভিডিওতে tap করলে controls দেখা যায় বা লুকায়, আর কয়েক সেকেন্ড পর নিজে থেকেই লুকিয়ে যায়।

৯.২ Player-এর তিনটি ঘটনা

কোডে কোথায় `VideoFragment.kt › listener যুক্ত করা` লাইন 51–53
`VideoFragment.kt › onVideoPrepared()` লাইন 102–112
`VideoFragment.kt › onVideoError()` লাইন 131–136
`VideoFragment.kt › onVideoCompleted()` লাইন 114–118

ভিডিওর ঠিকানা দেওয়ার পর VideoView সেটি asynchronous-ভাবে প্রস্তুত (prepare) করে; ততক্ষণ একটি loading indicator দেখা যায়। এরপর player তিনটি ঘটনার খবর দেয়, আর প্রতিটিই main thread-এর queue দিয়ে আসা একটি বার্তা (**২.৭**):

- **onPrepared:** ভিডিও প্রস্তুত বলে চিহ্নিত হয়, loading লুকানো হয়, এবং ViewModel-এ সংরক্ষিত অবস্থানে seek করা হয়। আগে চলছিল (resume করার কথা) হলে ভিডিও আবার চালু হয়; নইলে controls কিছুক্ষণ দেখানো হয়, যাতে play বাটন সহজে চোখে পড়ে।
- **onError:** Log-এ ত্রুটি লিখে loading লুকানো হয় এবং একটি বন্ধুসুলভ বার্তা দেখানো হয় ("Sorry, this video can't be played on this device.")। true ফেরত দেওয়া মানে ত্রুটিটি সামলানো হয়েছে, তাই

সিস্টেমের “Can't play this video” dialog আসে না।

- **onCompletion:** ভিডিও শেষ হলে ViewModel-এ অবস্থান ০ করা হয় এবং resume বন্ধ রাখা হয়, যাতে পরেরবার শুরু থেকে চলে; আর controls আবার দেখানো হয়, যাতে আবার চালানো এক tap দূরে থাকে।

৯.৩ থামানো ও মনে রাখা: onPause-এ কেন

কোডে কোথায় `VideoFragment.kt` › **`onPause()`** লাইন 67–82 | `VideoFragment.kt` › **`onStop()`** লাইন 84–89 | `VideoFragment.kt` › **`onStart()`** লাইন 61–65

স্ক্রিন যখনই চলে যেতে শুরু করে (rotation, Home চাপা, বা অন্য drawer item), `VideoFragment`-এর onPause দুটি কাজ করে: ভিডিও প্রস্তুত থাকলে বর্তমান অবস্থান ও “আবার চালাতে হবে কি না” ViewModel-এ রাখে, তারপর ভিডিও pause করে।

- **আবার চালানোর শর্ত:** ভিডিও চলছিল **এবং** `isChangingConfigurations` সত্য লাইন 78। অর্থাৎ শুধু rotation হলে নতুন স্ক্রিনে ভিডিও নিজে থেকেই আবার চলতে থাকে; কিন্তু ব্যবহারকারী সত্যিই স্ক্রিন ছেড়ে গেলে ফিরে এসে দেখবেন ভিডিও থেমে একই জায়গায় অপেক্ষা করছে।
- **onStop নয় কেন:** সাম্প্রতিক Android-এ onStop চলার আগেই VideoView-এর surface, আর তার সঙ্গে MediaPlayer, ছেড়ে দেওয়া হয়। তখন অবস্থান পড়লে ০ পাওয়া যেত। onPause-এর সময় player তখনও জীবিত।
- **onStop-এ যা হয়:** “প্রস্তুত” চিহ্নটি মুছে ফেলা হয়। ব্যাকগ্রাউন্ডে VideoView তার player ছেড়ে দেয়; ফিরে এলে ভিডিও আবার prepare হয় এবং onPrepared আবার আসে। তাই onStart প্রস্তুত না থাকলে loading দেখায়।

৯.৪ VideoViewModel: অবস্থান কোথায় থাকে

কোডে কোথায় `VideoViewModel.kt` › **`VideoViewModel`** লাইন 14–37 | `VideoViewModel.kt` › **`savePlayback()`** লাইন 25–28

অবস্থান VideoView নিজে মনে রাখতে পারে না: rotation-এ view ধ্বংস হয়, আর ব্যাকগ্রাউন্ডে গেলে তার player মুক্ত হয়ে যায়। তাই অবস্থান (মিলিসেকেন্ডে) ও “চলছিল কি না” থাকে `VideoViewModel`-এর SavedStateHandle-এ, যা rotation ও process death দুটোতেই টিকে থাকে (**৫.২**)।

অডিওর মতো এখানে player-টি ViewModel-এ রাখা হয়নি কেন? কারণ VideoView একটি view, এবং তার player স্ক্রিনের surface-এর সঙ্গে বাঁধা; view চলে গেলে player-ও যায়। তাই এখানে পুরো player নয়, শুধু “কোথায় ছিলাম” আর “চলছিল কি না”, এই দুটি তথ্য সংরক্ষণ করা হয়, এবং নতুন player প্রস্তুত হলে সেখান থেকে শুরু হয়।

৯.৫ Controls ও drawer-এর সমন্বয়

কোডে কোথায় `VideoFragment.kt` › **`hideControlsWhileDrawerMoves`** লাইন 147–151 | `VideoFragment.kt` › **`showControlsBriefly()`** লাইন 126–128 | `VideoFragment.kt` › **`onDestroyView()`** লাইন 91–100

- **Drawer নড়লে controls লুকানো:** MediaController Activity-র ওপরে নিজের আলাদা window-তে আঁকা হয়। Drawer খুললে drawer controls-এর নিচ দিয়ে চলে যেত, আর controls drawer-এর ওপরে ভেসে থাকত। তাই একটি DrawerListener drawer-এর অবস্থা স্থির (idle) থেকে বদলালেই, অর্থাৎ টানা শুরু হলেই, controls লুকিয়ে দেয়। এটি observer pattern: drawer তার অবস্থার পরিবর্তন জানায়, আর Fragment শুনে প্রতিক্রিয়া দেখায়।
- **স্থায়ীভাবে দেখানো হয় না কেন:** controller একটি focusable window; এটি খোলা থাকলে toolbar বা hamburger কীবোর্ড focus পায় না। তাই controls স্থায়ী না করে ডিফল্ট সময়সীমা (কয়েক সেকেন্ড) রাখা হয়েছে। TalkBack চালু থাকলে MediaController নিজেই সময়সীমা উপেক্ষা করে controls দেখিয়ে রাখে।
- **পরিচ্ছন্নভাবে বন্ধ:** view ধ্বংসের সময় drawer listener সরানো হয় (নইলে Activity-র drawer ধ্বংস হওয়া Fragment-কে ধরে রাখত), controller লুকানো হয় (যাতে তার window leak না হয়), এবং `stopPlayback()` দিয়ে MediaPlayer মুক্ত করা হয়।

১০ ফিচার D: Audio

সংশ্লিষ্ট ফাইল (`audio` প্যাকেজ): `AudioPlayerViewModel` · `AudioFragment` · `util/TimeFormat.kt` · `res/layout/fragment_audio.xml` (ও `layout-land`) · `res/raw/sample_audio.mp3`

অডিও স্ক্রিনে MediaPlayer সরাসরি ব্যবহার করা হয়েছে, কোনো তৈরি controller ছাড়া। তাই এখানে player-এর state machine, UI-এর state এবং lifecycle, তিনটিই নিজেদের হাতে সামলাতে হয়। Message queue-এর ব্যবহারও এখানে সবচেয়ে স্পষ্ট (**২.৮**)।

১০.১ MediaPlayer একটি state machine

কোডে কোথায় `AudioPlayerViewModel.kt` › **`state-এর পথ (মন্তব্য)`** লাইন 35–43

MediaPlayer সবসময় নির্দিষ্ট কয়েকটি অবস্থার (state) একটিতে থাকে, আর প্রতিটি method শুধু নির্দিষ্ট অবস্থায় ডাকা যায়। ভুল অবস্থায় ডাকলে IllegalStateException বা ত্রুটি হয়। এই অ্যাপ যে পথে চলে:

বর্তমান অবস্থা	কাজ বা ঘটনা	পরের অবস্থা
Idle (নতুন তৈরি)	`setDataSource()`	Initialized
Initialized	`prepareAsync()`	Preparing
Preparing	onPrepared (player জানায়)	Prepared
Prepared, Paused বা PlaybackCompleted	`start()`	Started
Started	`pause()`	Paused
Started বা Paused	`stop()`	Stopped
Stopped	`prepareAsync()`	Preparing, তারপর Prepared
Started	ট্র্যাক শেষ	PlaybackCompleted
যেকোনো অবস্থা	`release()`	End

সবচেয়ে গুরুত্বপূর্ণ নিয়ম: Stopped থেকে সরাসরি start করা যায় না, আগে আবার prepare করতে হয়। তাই অ্যাপের `stop()` নিজেই শেষে `prepareAsync()` ডাকে।

১০.২ Player কেন ViewModel-এ

কোডে কোথায় `AudioPlayerViewModel.kt` › **`player`** লাইন 50
`AudioPlayerViewModel.kt` › **`init`** লাইন 53–70 `AudioPlayerViewModel.kt` › **`onCleared()`** লাইন 141–144

MediaPlayer তৈরি হয় `AudioPlayerViewModel`-এর ভেতরে, Fragment-এ নয়। Rotation-এ Fragment ও তার view নতুন করে তৈরি হয়, কিন্তু ViewModel ও তার player একই থাকে; তাই গান থামে না, এক মুহূর্তের বিরতি ছাড়াই চলতে থাকে। স্ক্রিন স্থায়ীভাবে চলে গেলে `onCleared()`-এ progress-এর কাজ বাতিল করে `release()` ডাকা হয়, যা decoder ও অডিও রিসোর্স মুক্ত করে।

ViewModel-টি AndroidViewModel, কারণ raw resource খুলতে Application context লাগে; Activity-র context রাখলে rotation-এ পুরোনো Activity leak হতো। ViewModel তৈরির সময় player প্রস্তুত করার ধাপগুলো:

১. AudioAttributes (`USAGE_MEDIA`, `CONTENT_TYPE_MUSIC`) দিয়ে সিস্টেমকে জানানো যে এটি গান বা মিডিয়া, ফলে media volume ও অডিওর গন্তব্য (স্পিকার, হেডফোন) ঠিক থাকে। এটি prepare-এর আগেই দিতে হয় লাইন 55–60 ।

২. `res/raw/sample_audio.mp3`-এর file descriptor খুলে `setDataSource()`, অর্থাৎ Idle থেকে Initialized লাইন 61–63 ।

৩. onPrepared, onCompletion ও onError listener যুক্ত করা লাইন 64–66 ।

৪. `prepareAsync()`, অর্থাৎ Initialized থেকে Preparing লাইন 69 ।

`prepareAsync()` সঙ্গে সঙ্গে ফিরে আসে; ফাইল প্রস্তুত হলে player main thread-এ onPrepared পাঠায়। এটি asynchronous request/reply: অনুরোধ পাঠানো হয়, উত্তর আসে পরে একটি বার্তা হিসেবে। সাধারণ `prepare()` ব্যবহার করলে প্রস্তুত হওয়া পর্যন্ত main thread আটকে থাকত।

১০.৩ State উপরে ওঠে, command নিচে নামে

কোডে কোথায় `AudioPlayerViewModel.kt › AudioUiState` লাইন 20–26
`AudioPlayerViewModel.kt › togglePlayPause(), play(), pause()` লাইন 72–88
`AudioFragment.kt › onViewCreated()` লাইন 30–44 `AudioFragment.kt › render()` লাইন 76–103

পুরো স্ক্রিনের অবস্থা একটি immutable data class-এ রাখা হয়: `AudioUiState`, যার পাঁচটি ফিল্ড বলে দেয়: প্রস্তুত কি না, চলছে কি না, বর্তমান অবস্থান, মোট সময় এবং ত্রুটি আছে কি না। এটি একটি StateFlow-এ প্রকাশ হয়, আর ViewModel-ই এর একমাত্র লেখক।

- **Command নিচে (UI থেকে ViewModel):** Fragment ব্যবহারকারীর কাজকে ViewModel-এর method-এ রূপ দেয়: `togglePlayPause()`, `stop()`, `seekTo()`। Fragment নিজে কখনো player ছোঁয় না।
- **State উপরে (ViewModel থেকে UI):** Fragment `repeatOnLifecycle(STARTED)`-এর ভেতরে StateFlow collect করে এবং প্রতিটি নতুন state-এ `render()` দিয়ে পুরো UI আঁকে। এই StateFlow observer pattern-এ একটি retained topic-এর মতো কাজ করে (**৩.৭**)।
- **পূর্বশর্ত যাচাই:** প্রতিটি command আগে state দেখে নেয়: play শুধু প্রস্তুত এবং না-চলা অবস্থায়, pause শুধু চলা অবস্থায়, stop ও seek শুধু প্রস্তুত অবস্থায়। এতে state machine-এর ভুল অবস্থায় কোনো method ডাকা হয় না।
- **Immutable হালনাগাদ:** প্রতিবার পুরোনো state কপি করে শুধু দরকারি ফিল্ড বদলানো হয়; StateFlow নতুন মানটি সব collector-কে পৌঁছে দেয়।

এই ধরনকে **unidirectional data flow** বলে: সত্যের উৎস একটিই (ViewModel-এর state), আর UI শুধু সেই state-এর প্রতিচ্ছবি। `render()` state থেকেই সব ঠিক করে: Play/Pause বাটনের লেখা ও আইকন; বাটন সক্রিয় কি না (প্রস্তুত না হলে Play নিষ্ক্রিয়, আর Stop শুধু তখনই সক্রিয় যখন গান চলছে বা অবস্থান শূন্যের বেশি); slider; সময়; এবং অবস্থার লেখা: Preparing the track…, Ready to play, Playing, Paused, বা ত্রুটির বার্তা।

১০.৪ Progress: coroutine-এর একটি লুপ

কোডে কোথায় `AudioPlayerViewModel.kt › startProgressUpdates()` লাইন 125–133
`AudioPlayerViewModel.kt › stopProgressUpdates()` লাইন 135–138
`AudioPlayerViewModel.kt › PROGRESS_INTERVAL_MS` লাইন 148

গান চলার সময় slider ও সময় এগোতে হবে, কিন্তু MediaPlayer নিজে থেকে অবস্থান জানায় না; জিজ্ঞেস করে নিতে হয়। তাই গান চালু হলে ViewModel-এর scope-এ একটি coroutine চালু হয়: যতক্ষণ সক্রিয়, ততক্ষণ player-এর বর্তমান অবস্থান state-এ বসায়, তারপর ২০০ মিলিসেকেন্ড অপেক্ষা করে; অর্থাৎ সেকেন্ডে প্রায় পাঁচবার।

এই অপেক্ষা thread আটকায় না। Main dispatcher-এ `delay()` আসলে একটি delayed বার্তা, যা ২০০ মিলিসেকেন্ড পরে queue-তে চলার উপযোগী হয় (**২.৫**); মাঝের সময়ে main thread touch, drawing ইত্যাদি অন্য কাজ করে। Messaging-এর ভাষায় এটি একটি periodic producer। Pause, stop, ট্র্যাক শেষ বা ত্রুটি হলে job বাতিল হয়, আর `onCleared()`-এও।

১০.৫ Slider ও সময়ের লেখা

কোডে কোথায় `AudioFragment.kt › setUpSeekSlider()` লাইন 55–74
`AudioFragment.kt › render() (slider অংশ)` লাইন 84–92 `TimeFormat.kt › formatMmSs()` লাইন 14–19

- **Slider-এর সীমা:** Material Slider-এর সর্বোচ্চ মান হলো মোট সময় (মিলিসেকেন্ডে), তবে কমপক্ষে ১; কারণ Slider-এ সর্বোচ্চ মান অবশ্যই সর্বনিম্ন মানের (০) চেয়ে বড় হতে হয়, নইলে Slider exception ছোড়ে। বর্তমান মানও সবসময় সীমার ভেতরে আটকে রাখা হয়।
- **আঙুলের সঙ্গে "যুদ্ধ" না করা:** slider টানার সময় প্রতি ২০০ মিলিসেকেন্ডের progress আপডেট slider-কে পুরোনো অবস্থানে টেনে নিত। তাই `isUserSeeking` flag: আঙুল রাখার মুহূর্ত থেকে ছাড়া পর্যন্ত state থেকে slider বসানো হয় না, শুধু সময়ের লেখা আঙুলের মান দেখায়। আঙুল ছাড়লে তবেই seek।
- **কীবোর্ড ও accessibility:** এসব উপায়ে মান বদলালে কোনো touch পর্ব থাকে না, তাই সঙ্গে সঙ্গে seek করা হয়। বদলটি ব্যবহারকারীর, নাকি কোডের, তা listener-এর `fromUser` থেকে বোঝা যায়।
- **mm:ss:** `formatMmSs()` মিলিসেকেন্ডকে মিনিট:সেকেন্ডে রূপ দেয়; যেমন ৬৫,০০০ মিলিসেকেন্ড হয় "01:05"। ঋণাত্মক মান ০ ধরা হয়; এক ঘণ্টার বেশি হলে মিনিট গুনতেই থাকে ("75:00")। `Locale.ROOT` ব্যবহার করায় ফোনের যেকোনো ভাষার সেটিংয়ে একই অঙ্ক দেখায়। Slider টানার সময় ভেসে ওঠা লেবেলও একই function ব্যবহার করে।

১০.৬ Stop, ট্র্যাক শেষ ও ত্রুটি

কোডে কোথায় `AudioPlayerViewModel.kt › stop()` লাইন 91–97
`AudioPlayerViewModel.kt › onCompleted()` লাইন 111–115
`AudioPlayerViewModel.kt › onError()` লাইন 117–122

- **Stop:** player থামে (Stopped অবস্থা), progress বন্ধ হয়, state-এ "প্রস্তুত নয়, চলছে না, অবস্থান ০" বসে, তারপর `prepareAsync()`। প্রস্তুত হলে onPrepared আবার "প্রস্তুত" করে দেয়। ফলে Stop মানে "থামো

এবং শুরুতে ফিরে যাও"।

- **ট্র্যাক শেষ:** progress বন্ধ, অবস্থান ০-তে seek, "চলছে না" ও অবস্থান ০; আবার Play চাপলেই শুরু থেকে বাজে।
- **ত্রুটি:** Log, progress বন্ধ, state-এ ত্রুটি চিহ্নিত, এবং true ফেরত, যাতে এর পরে onCompletion-ও না ডাকা হয় (false ফেরত দিলে MediaPlayer onCompletion-ও ডাকত)।

১০.৭ অডিও ও ভিডিও কখনো একসঙ্গে বাজে না

কোডে কোথায় `AudioFragment.kt` › **`onStop()`** লাইন 46–53
`AudioPlayerViewModel.kt` › **`pause()`** লাইন 83–88

সমস্যা: অন্য drawer item-এ যাওয়ার সময় Navigation অডিও স্ক্রিনের ViewModel বাঁচিয়ে রাখে (**৬.২**)। Player ViewModel-এ, তাই কিছু না করলে Video স্ক্রিনে গেলেও পেছনে গান বাজতে থাকত এবং ভিডিওর শব্দের সঙ্গে মিশে যেত।

সমাধান: `AudioFragment`-এর onStop-এ, যদি `isChangingConfigurations` সত্য না হয় (অর্থাৎ এটি rotation নয়, সত্যিই চলে যাওয়া: অন্য drawer item, Home ইত্যাদি), তবে ViewModel-কে pause করতে বলা হয়।

ফলাফল: rotation-এ গান চলতেই থাকে; স্ক্রিন ছাড়লে থেমে যায়; ফিরে এসে থামা জায়গা থেকেই Play করা যায়। আর ভিডিও স্ক্রিনও নিজের onPause-এ থামে। তাই দুটি একসঙ্গে বাজার কোনো সুযোগ থাকে না।

১১ সব স্ক্রিনে প্রযোজ্য বিষয়

কিছু আচরণ কোনো একটি ফিচারের নয়, পুরো অ্যাপের। এগুলোর বেশিরভাগ আগের অধ্যায়গুলোতে টুকরো টুকরো এসেছে; এখানে একসঙ্গে দেখা যাক।

১১.১ Rotation: কোন স্ক্রিন কী ধরে রাখে

অ্যাপ `android:configChanges` ব্যবহার করে না, তাই rotation-এ সিস্টেম প্রতিটি স্ক্রিন নতুন করে তৈরি করে এবং landscape-এর উপযোগী resource লোড করে (battery status ও audio স্ক্রিনের আলাদা landscape layout আছে)। তবু কোনো তথ্য হারায় না (**৫.২**):

স্ক্রিন	যা টিকে থাকে	যেভাবে ও কোডে কোথায়
Broadcast স্ক্রিন ১	Spinner-এর নির্বাচন	SavedStateHandle `BroadcastViewModel.kt` › **`selectedMode`** লাইন 16–17
Activity 2 (custom)	টাইপ করা লেখা	Text field-এর নিজস্ব saved state (id থাকায়); আলাদা কোড লাগেনি
Activity 2 (battery)	সর্বশেষ ব্যাটারির তথ্য	ViewModel-এর LiveData ও sticky broadcast `BatteryViewModel.kt` › **`BatteryViewModel`** লাইন 14–17
Activity 3 (custom)	প্রাপ্ত বার্তা; "পাঠানো হয়েছে" flag	SavedStateHandle `CustomReceiverViewModel.kt` › **`CustomReceiverViewModel`** লাইন 16–38
Image Scale	ডাউনলোড করা ছবি; zoom ও দৃশ্যমান অংশ	StateFlow ও view-এর saved state `ImageScaleViewModel.kt` › **`uiState`** লাইন 35–36 `ZoomableImageView.kt` › **`onSave/onRestoreInstanceState()`** লাইন 268–291
Video	অবস্থান; চলছিল কি না	SavedStateHandle `VideoViewModel.kt` › **`savePlayback()`** লাইন 25–28
Audio	পুরো player; গান চলতেই থাকে	ViewModel-এ player `AudioPlayerViewModel.kt` › **`player`** লাইন 50
Drawer	খোলা না বন্ধ	DrawerLayout-এর saved state; Back callback মিলিয়ে নেওয়া `MainActivity.kt` › **`onRestoreInstanceState()`** লাইন 52–57

১১.২ Theme, resource ও accessibility

- **Light ও dark theme:** অ্যাপ Material 3-এর DayNight theme ব্যবহার করে, তাই ফোনে dark mode চালু করলে সব স্ক্রিন নিজে থেকেই গাঢ় রঙে চলে যায়।
- **সব লেখা ও মাপ resource-এ:** লেখা `strings.xml`-এ, মাপ `dimens.xml`-এ (landscape-এর জন্য `values-land`); layout-এ কোনো hard-coded লেখা বা মাপ নেই। ফলে অনুবাদ করা বা ভিন্ন আকারের

স্ক্রিনে মানিয়ে নেওয়া সহজ। সংখ্যার সঙ্গে সঠিক বচনের জন্য plurals resource ব্যবহার করা হয়।

- **Accessibility:** প্রতিটি touch target কমপক্ষে ৪৮dp; ছবি, ভিডিও, ব্যাটারির indicator ও slider-এর content description আছে; zoom view-এ একক tap-এ `performClick()` লাইন 182–185; TalkBack চালু থাকলে MediaController controls দেখিয়ে রাখে।
- **Edge-to-edge:** প্রতিটি স্ক্রিন দুই orientation-এই সিস্টেম বার ও ক্যামেরার খাঁজ এড়িয়ে চলে (**৬.৫**)।

১২ দ্রুত পুনরালোচনা

পরীক্ষা বা viva-র আগে এক নজরে দেখে নেওয়ার জন্য দুই বার্তা ব্যবস্থা ও প্রতিটি ফিচারের মূল কথা, আর শেষে এই নির্দেশিকায় ব্যবহৃত পরিভাষার একটি তালিকা।

১২.১ Message Queue ও Pub/Sub-এর মূল কথা

Message Queue

- Producer বার্তা queue-তে রাখে, consumer একটি একটি করে তোলে; সাধারণত FIFO।
- প্রতিটি বার্তা ঠিক একজন consumer প্রক্রিয়া করে (point-to-point); consumer ব্যস্ত থাকলে বার্তা অপেক্ষা করে, হারায় না।
- উদাহরণ: রেস্তোরাঁর অর্ডার স্লিপ, ব্যাংকের টোকেন, অনলাইন দোকানের email queue।
- Android-এ: main thread-এর Looper (consumer) ও MessageQueue; Handler বার্তা রাখে। Consumer একজন, তাই কোনো callback ধীর হলে ANR।
- এই অ্যাপে: touch event, lifecycle callback, `onReceive()`, onPrepared, `delay(200)`-এর লুপ, animation frame।

Publish/Subscribe

- Publisher topic-এ বার্তা দেয়; broker topic-এর সব subscriber-কে একটি করে কপি দেয় (fan-out); publisher subscriber-দের চেনে না।
- দেরিতে আসা subscriber: non-durable হলে কিছু পায় না (রেডিও), retained হলে শেষ বার্তা পায় (নোটিশ বোর্ড), durable হলে সব পায় (inbox)।
- Android-এ: Intent = বার্তা, action = topic, IntentFilter = filter, BroadcastReceiver = subscriber, ActivityManagerService = broker।
- এই অ্যাপে: custom broadcast (non-durable, তাই আগে register, পরে send); ব্যাটারি (sticky, অর্থাৎ retained)।
- ভেতরের pub/sub: LiveData ও StateFlow (observer pattern), যা retained topic-এর মতো আচরণ করে।

১২.২ প্রতিটি ফিচারের মূল কথা

Navigation Drawer

- একটি `MainActivity`; চারটি স্ক্রিন চারটি Fragment, যা NavHostFragment-এ দেখানো হয়।
- Menu item-এর id = destination-এর id; এই মিল দিয়েই NavigationUI স্ক্রিন খোলে ও item হাইলাইট করে।
- Menu থেকে AppBarConfiguration, তাই চারটিই top-level এবং সবখানে hamburger।
- Back callback শুধু drawer খোলা থাকলে সক্রিয়; তখন Back আগে drawer বন্ধ করে।

Custom broadcast

- টেক্সট Activity 2 থেকে Activity 3-এ যায় explicit Intent-এ (point-to-point, নিশ্চিত delivery)।
- Activity 3: onStart-এ register (`RECEIVER_NOT_EXPORTED`), তারপর `sendBroadcast()` (`setPackage()`-সহ); `onReceive()` main thread-এ চলে, ViewModel হয়ে UI; onStop-এ unregister।
- Activity 2 থেকে broadcast করলে বার্তা হারাত, কারণ সাধারণ broadcast non-durable এবং তখন receiver ছিলই না।
- SavedStateHandle-এর flag দিয়ে rotation-এ duplicate ঠেকানো হয়; প্রাপ্ত বার্তাও সেখানেই থাকে।

Battery broadcast

- `ACTION_BATTERY_CHANGED` sticky (সর্বশেষ মান সঙ্গে সঙ্গে পাওয়া যায়) ও protected (শুধু সিস্টেম পাঠায়), এবং শুধু কোডে register করা receiver পায়।
- LiveData-র onActive-এ register, onInactive-এ unregister, তাই শুধু স্ক্রিন দৃশ্যমান থাকলেই শোনা হয়।
- শতাংশ = level × ১০০ ÷ scale; scale শূন্য বা level অনুপস্থিত হলে "—"।

Image Scale

- Coil 3 দিয়ে মূল আকারে (`Size.ORIGINAL`) ছবি; ViewModel-এর StateFlow-এ Loading, Success বা Error; Retry।
- scaleType MATRIX; zoom = বর্তমান scale ÷ fit scale, সীমা ১x থেকে ৫x।
- ScaleGestureDetector-এর focal point ঘিরে scale, তাই আঙুলের নিচের বিন্দু স্থির থাকে।
- এক আঙুলে pan, কিনারায় ফাঁক না রাখা, double-tap-এ ২.৫x বা রিসেট (২৫০ মিলিসেকেন্ডের animation)।
- Zoom ও কেন্দ্রবিন্দু ভগ্নাংশে সংরক্ষণ, তাই rotation-এর পরও একই দৃশ্য।

Video

- VideoView (ভেতরে MediaPlayer ও surface) এবং MediaController; ফাইল `res/raw`-এ।
- onPause-এ অবস্থান রাখা হয় (onStop-এ দেরি হয়ে যায়); আবার চলে শুধু তখনই, যখন ভিডিও চলছিল এবং কারণটি rotation।
- ত্রুটিতে নিজস্ব বার্তা, সিস্টেম dialog নয়; drawer নড়লে controls লুকানো।

Audio

- MediaPlayer একটি state machine; stop-এর পর আবার `prepareAsync()` লাগে।
- Player ViewModel-এ, তাই rotation-এ গান চলে; `onCleared()`-এ release।
- একটি `AudioUiState` StateFlow: state উপরে ওঠে, command নিচে নামে।
- ২০০ মিলিসেকেন্ড পরপর coroutine-এ progress; slider টানার সময় কোড তার মান বদলায় না; onStop-এ pause (rotation ছাড়া)।

১২.৩ পরিভাষা

শব্দ	অর্থ
Message	একটি কাজ বা খবরের প্যাকেট: header ও body (payload)।
Producer ও Consumer	যে বার্তা queue-তে রাখে, আর যে queue থেকে তুলে প্রক্রিয়া করে।
Message queue	বার্তার সারি; প্রতিটি বার্তা একজন consumer ক্রম অনুযায়ী প্রক্রিয়া করে।
FIFO	First In, First Out: যে আগে এসেছে, সে আগে বের হয়।
Point-to-point	একটি বার্তা, একজন নির্দিষ্ট প্রাপক।
Competing consumers	একাধিক consumer একই queue থেকে কাজ ভাগ করে নেয়; কেউ একই বার্তা দুবার পায় না।
Acknowledgement (ack)	Consumer-এর "কাজ শেষ" জানানো; তবেই queue বার্তা মুছে ফেলে।
Load leveling	ভিড়ের সময়ের বার্তা queue-তে জমে থাকে, consumer নিজের গতিতে শেষ করে।
Publish/Subscribe	Publisher topic-এ বার্তা দেয়; broker সেটি সব subscriber-এর কাছে পৌঁছায়।
Topic	বার্তার শ্রেণির নাম; Android-এ Intent-এর action।
Broker	মাঝখানের ব্যবস্থা, যে subscription-এর তালিকা রাখে ও বার্তা পৌঁছায়; Android-এ ActivityManagerService।
Fan-out	একটি বার্তার কপি একসঙ্গে অনেক subscriber-এর কাছে যাওয়া।
Non-durable বার্তা	শুধু তখন উপস্থিত subscriber পায়; দেরিতে আসা কেউ পায় না।
Retained বার্তা	Broker শেষ বার্তাটি রেখে দেয়; নতুন subscriber সঙ্গে সঙ্গে পায়।

শব্দ	অর্থ
Observer pattern	একই প্রোগ্রামের ভেতরে broker-ছাড়া pub/sub; যেমন LiveData ও StateFlow।
Activity	একটি স্ক্রিন এবং তার lifecycle-এর মালিক Android component।
Fragment	Activity-র ভেতরের পুনঃব্যবহারযোগ্য স্ক্রিন-অংশ, যার নিজস্ব lifecycle ও view আছে।
Intent	Android component-গুলোর মধ্যে পাঠানো বার্তা: action, extras এবং প্রয়োজনে গন্তব্য।
Explicit Intent	গন্তব্য class নির্দিষ্ট করে দেওয়া Intent; একটি point-to-point বার্তা।
Broadcast	একটি action-সহ Intent, যা সিস্টেম মিলে যাওয়া সব receiver-এর কাছে পৌঁছায়।
BroadcastReceiver	Broadcast গ্রহণকারী component; এর `onReceive()` main thread-এ চলে।
IntentFilter	Receiver কোন action গ্রহণ করবে তার শর্ত; subscription filter।
Sticky broadcast	যে broadcast-এর শেষ কপি সিস্টেম রেখে দেয়; নতুন receiver সঙ্গে সঙ্গে পায়।
Protected broadcast	যে broadcast শুধু সিস্টেম পাঠাতে পারে।
Looper ও MessageQueue	Main thread-এর লুপ ও তার queue; বার্তা একটি একটি করে নিয়ে চালায়।
Handler	Main thread-এর queue-তে বার্তা রাখার উপায় (`post()`, `sendMessageDelayed()`)।
ANR	Application Not Responding; main thread দীর্ঘক্ষণ আটকে থাকলে সিস্টেমের সতর্কবার্তা।
Lifecycle	Activity বা Fragment-এর তৈরি থেকে ধ্বংস পর্যন্ত অবস্থাগুলোর ধারা।
Configuration change	Rotation, ভাষা বা theme বদল; সাধারণত Activity নতুন করে তৈরি হয়।
ViewModel	Configuration change পেরিয়েও বেঁচে থাকা UI-ডেটার ধারক।
SavedStateHandle	ViewModel-এর key-value store; process death-এর পরও মান ফিরে আসে।
LiveData	Lifecycle-সচেতন observable মান; শুধু active observer-কে জানায়।
StateFlow	Coroutine-ভিত্তিক observable, যা সবসময় একটি সর্বশেষ মান ধরে রাখে।
Coroutine	Thread না আটকে থামতে ও আবার চলতে পারে এমন হালকা কাজের একক।
Matrix	বিন্দুকে scale ও translate করে এক স্থানাঙ্ক থেকে আরেকটিতে নেওয়ার ৩×৩ রূপান্তর।
Focal point	Pinch-এর দুই আঙুলের মধ্যবিন্দু; zoom এই বিন্দুকে ঘিরে হয়।
MediaPlayer	অডিও ও ভিডিও চালানোর Android class; একটি state machine।
Window insets	সিস্টেম বার, ক্যামেরার খাঁজ বা কীবোর্ড স্ক্রিনের কতটা ঢেকেছে তার মাপ।